Zadanie 4: obliczenia ewolucyjne

Autorzy: Anna Moroń, Kalina Rączka, Emilia Sarna

Termin realizacji: 4 maja 2026

Zadanie do oddania przez MS Teams. Do oddania: kod oraz krótkie sprawozdanie w PDF (można na przykład przy użyciu `quarto render notebook.ipynb --to pdf`).

Na 3.0

Do realizacji:

1. Zmodyfikuj przykład "iterative method" z notatnika `tsp_lazy_constraints.ipynb` tak, aby rozwiązywał problem podróżującego konsumenta. Przykładowe problemy generuje poniższa funkcja `generate_random_tpp`. Przyjmij, że wszystkie miasta muszą być odwiedzone niezależnie od tego czy dokonany tam będzie zakup czy też nie. W rozwiązaniu wypisz w którym mieście kupione będą poszczególne przedmioty oraz narysuj trasę.
2. Zmodyfikuj analogicznie kod z notatnika `tsp-genetic.ipynb` tak aby rozwiązywał ten sam problem. Porównaj wydajność (czas obliczeń i wartość funkcji kosztu) dla dziewięciu losowych problemów, po trzy dla rozmiarów (n=50, k=20), (n=50, k=50) oraz (n=100, k=100). Dobierz parametry algorytmu genetycznego tak, aby czas działania nie przekraczał w żadnym przypadku minuty.

Na 4.0

Do realizacji:

1. Punkty z zadania na 3.0.

2. Dodaj do problemu wagę przedmiotów, powiększającą koszt rozwiązania o długość trasy od miejsca zakupu do końca trasy pomnożoną przez wagę przedmiotu i stały współczynnik dobrany tak, aby dla przedmiotu średniej wagi jego transport między najbardziej odległymi miastami (w linii prostej) wynosił około 1% ceny. Nie dodawaj tego składnika do funkcji kosztu zaimplementowanej w JuMP, ale uwzględnij go w algorytmie genetycznym. Porównaj jakość rozwiązań uzyskanych przez oba algorytmy z uwzględnieniem wagi i bez.
3. Zaimplementuj dodatkowo krzyżowanie PMX i porównaj wydajność z tym opartym o inwersję permutacji.

Na 5.0

Do realizacji:

1. Punkty z zadania na 4.0.
2. Zmodyfikuj problem tak, aby konsument nie musiał odwiedzać miast w których niczego nie kupuje.
3. Dodaj dodatkowo do porównania zmodyfikowany dla problemu podróżującego konsumenta przykład "lazy constraints" z notatnika `tsp_lazy_constraints.ipynb`. Dane do problemu powinny być generowane tak, aby konsument w zwracanym rozwiązaniu odwiedzał przynajmniej 3 miasta i omijał przynajmniej 2.

```
# import Pkg; Pkg.add("JuMP")

using JuMP
import GLPK
import Random
import Plots

function generate_random_tpp(n, k; random_seed=1)
    rng = Random.MersenneTwister(random_seed)
    # distance matrix
    X = 100 * rand(rng, n)
    Y = 100 * rand(rng, n)
    d = [sqrt((X[i] - X[j])^2 + (Y[i] - Y[j])^2) for i in 1:n, j in 1:n]

    cost_item_city = 50 .+ 100 .* rand(rng, k, n) # cost_item_city[num_item,
 ↪  num_city]

    return X, Y, d, cost_item_city
end

n = 100
```

```julia
k = 100
X, Y, d, cost_item_city = generate_random_tpp(n, k)
```

```
([9.913970137863682, 70.19797138879542, 50.3261785841856, 87.58412053070398,
 ↪  95.34654118744876, 50.7810571056071, 78.97511635624403,
 ↪  7.125413261100788, 13.837807897217225, 39.31891799217675  …
 ↪  84.87369607977678, 61.680928138712, 5.665730912653899,
 ↪  15.622563304879634, 36.90767228785501, 70.07597765092129,
 ↪  79.43901471209098, 46.482254570311675, 68.59072330642508,
 ↪  86.69884288310024], [96.78179466896867, 56.23453714649542,
 ↪  67.44638756669107, 7.1115103002265645, 92.78034391338332,
 ↪  34.57366887562756, 76.53412034001651, 33.078576899782796,
 ↪  62.27235533684083, 31.31072581673351  …  1.4463814325218927,
 ↪  42.253985947804495, 53.81635009641501, 76.50117040708963,
 ↪  27.74238915740479, 60.20183753580153, 19.81346291572821,
 ↪  90.60507365183767, 31.10234142135033, 21.085230265206945], [0.0
 ↪  72.65150307747324 … 88.07242440917794 107.82340444001487;
 ↪  72.65150307747324 0.0 … 25.183536454701592 38.829789264256306; … ;
 ↪  88.07242440917794 25.183536454701592 … 0.0 20.69411777577625;
 ↪  107.82340444001487 38.829789264256306 … 20.69411777577625 0.0],
 ↪  [109.5644843458705 111.42012523610694 … 79.47407758024632
 ↪  64.08473004215529; 131.92333614539837 130.45622190341587 …
 ↪  103.71208777471574 64.76470863461792; … ; 76.45999464961106
 ↪  75.17527827579866 … 140.35633239834164 110.40360118016744;
 ↪  99.40003490860023 112.53849121263079 … 72.8900549827469
 ↪  63.00390343599385])
```

Na 3.0

Iterative method dla problemu podróżującego konsumenta (JuMP)

Zmodyfikowano model iteracyjny dodając zmienne `y[item, city]`, decydujące w którym mieście zostanie kupiony dany przedmiot. Funkcja celu sumuje długość trasy i koszty zakupu. Trasa jest budowana tak, żeby odwiedzić wszystkie miasta (ograniczenie `sum(x[i,:]) == 2`), a podtrasy są eliminowane iteracyjnie.

```julia
using Printf

function selected_edges(x::AbstractMatrix, n)
    return Tuple{Int,Int}[(i, j) for i in 1:n, j in 1:n if x[i, j] > 0.5]
```

```julia
end

function subtour(edges::Vector{Tuple{Int,Int}}, n)
    shortest_subtour, unvisited = collect(1:n), Set(collect(1:n))
    while !isempty(unvisited)
        this_cycle, neighbors = Int[], unvisited
        while !isempty(neighbors)
            current = pop!(neighbors)
            push!(this_cycle, current)
            if length(this_cycle) > 1
                pop!(unvisited, current)
            end
            neighbors = [j for (i, j) in edges if i == current && j in
 ↪  unvisited]
        end
        if length(this_cycle) < length(shortest_subtour)
            shortest_subtour = this_cycle
        end
    end
    return shortest_subtour
end

subtour(x::AbstractMatrix{Float64}) = subtour(selected_edges(x, size(x, 1)),
 ↪  size(x, 1))
subtour(x::AbstractMatrix{VariableRef}) = subtour(JuMP.value.(x))

function build_tpp_model(d, cost_item_city)
    n = size(d, 1)
    k = size(cost_item_city, 1)
    model = Model(GLPK.Optimizer)
    @variable(model, x[1:n, 1:n], Bin, Symmetric)
    @variable(model, y[1:k, 1:n], Bin)
    @objective(model, Min,
        sum(d .* x) / 2
        +
        sum(cost_item_city[i, c] * y[i, c] for i in 1:k, c in 1:n))
    @constraint(model, [i in 1:n], sum(x[i, :]) == 2)
    @constraint(model, [i in 1:n], x[i, i] == 0)
    @constraint(model, [i in 1:k], sum(y[i, :]) == 1)
    return model
end
```

```julia
function plot_tpp_tour(X, Y, x_val::AbstractMatrix; title="")
    n = size(x_val, 1)
    plt = Plots.plot(; title=title)
    Plots.scatter!(plt, X, Y; legend=false, color=:red)
    for i in 1:n, j in 1:n
        if i < j && x_val[i, j] > 0.5
            Plots.plot!(plt, [X[i], X[j]], [Y[i], Y[j]]; legend=false,
 ↪  color=:blue)
        end
    end
    return plt
end
function plot_problem(X, Y, d; title="Drogi (niebieski → zielony → żółty →
 ↪  czerwony)")
    n = length(X)
    plt = Plots.plot(; title=title, legend=false, framestyle=:box,
 ↪  aspect_ratio=:equal)
    dmax = maximum(d)
    cmap = Plots.cgrad([:blue, :green, :yellow, :red])
    for i in 1:n, j in (i+1):n
        ratio = d[i, j] / dmax
        base = cmap[ratio]
        col = Plots.RGBA(Plots.red(base), Plots.green(base),
 ↪  Plots.blue(base), 0.15 + 0.55 * ratio)
        Plots.plot!(plt, [X[i], X[j]], [Y[i], Y[j]]; color=col,
 ↪  linewidth=0.6)
    end
    Plots.scatter!(plt, X, Y; color=:black, markersize=4,
 ↪  markerstrokecolor=:white)
    return plt
end

function print_problem_info(X, Y, d, cost_item_city; weights=nothing,
    max_cities=10, max_items=10, max_cost_cols=6)
    n = length(X)
    k = size(cost_item_city, 1)
    println("Liczba miast n = $n,  liczba przedmiotów k = $k")
    nz = d[d.>0]
    @printf "Odległości:    min=%.2f   średnia=%.2f   max=%.2f\n" minimum(nz)
     ↪  (sum(d) / (n * (n - 1))) maximum(d)
    @printf "Ceny:          min=%.2f   średnia=%.2f   max=%.2f\n"
     ↪  minimum(cost_item_city) (sum(cost_item_city) /
     ↪  length(cost_item_city)) maximum(cost_item_city)
```

```
    println("\nWspółrzędne miast (pierwsze $(min(max_cities, n))):")
    @printf "  %4s %10s %10s\n" "id" "X" "Y"
    for i in 1:min(max_cities, n)
        @printf "  %4d %10.2f %10.2f\n" i X[i] Y[i]
    end
    n > max_cities && println("  ... ($(n - max_cities) więcej)")

    println("\nCeny przedmiot × miasto (pierwsze $(min(max_items, k)) ×
     ↪  $(min(max_cost_cols, n))):")
    print("        ")
    for c in 1:min(max_cost_cols, n)
        @printf "%8s" "c$c"
    end
    println()
    for item in 1:min(max_items, k)
        @printf "  it%2d " item
        for c in 1:min(max_cost_cols, n)
            @printf "%8.2f" cost_item_city[item, c]
        end
        println()
    end
    if k > max_items || n > max_cost_cols
        println("  ...")
    end

    if weights !== nothing
        println("\nWagi przedmiotów (pierwsze $(min(max_items, k))):")
        @printf "  %4s %10s\n" "item" "weight"
        for i in 1:min(max_items, k)
            @printf "  %4d %10.3f\n" i weights[i]
        end
        k > max_items && println("  ... ($(k - max_items) więcej)")
    end
end
```

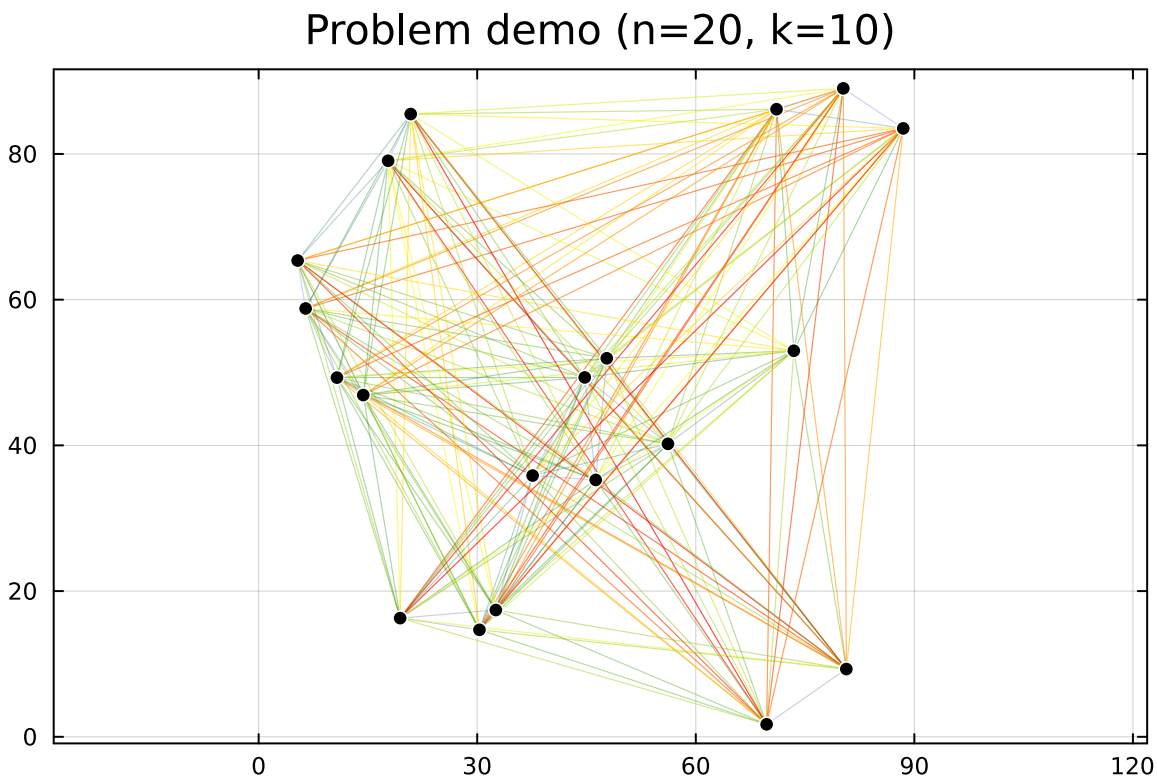
```
print_problem_info (generic function with 1 method)
```

```
function solve_tpp_iterative(d, cost_item_city; verbose=false)
    n = size(d, 1)
    model = build_tpp_model(d, cost_item_city)
    optimize!(model)
```

```julia
    @assert is_solved_and_feasible(model)
    total_time = solve_time(model)
    cycle = subtour(model[:x])
    while 1 < length(cycle) < n
        verbose && println("  Found cycle of length $(length(cycle))")
        S = [(i, j) for (i, j) in Iterators.product(cycle, cycle) if i < j]
        @constraint(model,
            sum(model[:x][i, j] for (i, j) in S) <= length(cycle) - 1)
        optimize!(model)
        @assert is_solved_and_feasible(model)
        total_time += solve_time(model)
        cycle = subtour(model[:x])
    end
    return model, total_time
end

# Mały problem
n_demo, k_demo = 20, 10
X_d, Y_d, d_d, cost_d = generate_random_tpp(n_demo, k_demo; random_seed=42)

println("--- Dane problemu (demo) ---")
print_problem_info(X_d, Y_d, d_d, cost_d)
display(plot_problem(X_d, Y_d, d_d; title="Problem demo (n=$n_demo,
 ↪  k=$k_demo)"))

println("\n--- Rozwiązanie iteratywne ---")
model_iter, t_iter = solve_tpp_iterative(d_d, cost_d; verbose=true)
println("Czas: $(round(t_iter, digits=3)) s")
println("Koszt całkowity: $(round(objective_value(model_iter), digits=3))")
```

```
--- Dane problemu (demo) ---
Liczba miast n = 20,  liczba przedmiotów k = 10
Odległości:    min=3.53   średnia=49.42   max=96.97
Ceny:          min=50.41   średnia=102.68   max=149.34

Współrzędne miast (pierwsze 10):
    id          X          Y
     1      71.08      86.14
     2       6.45      58.79
     3      47.78      51.95
     4      17.77      79.07
     5       5.35      65.37
```

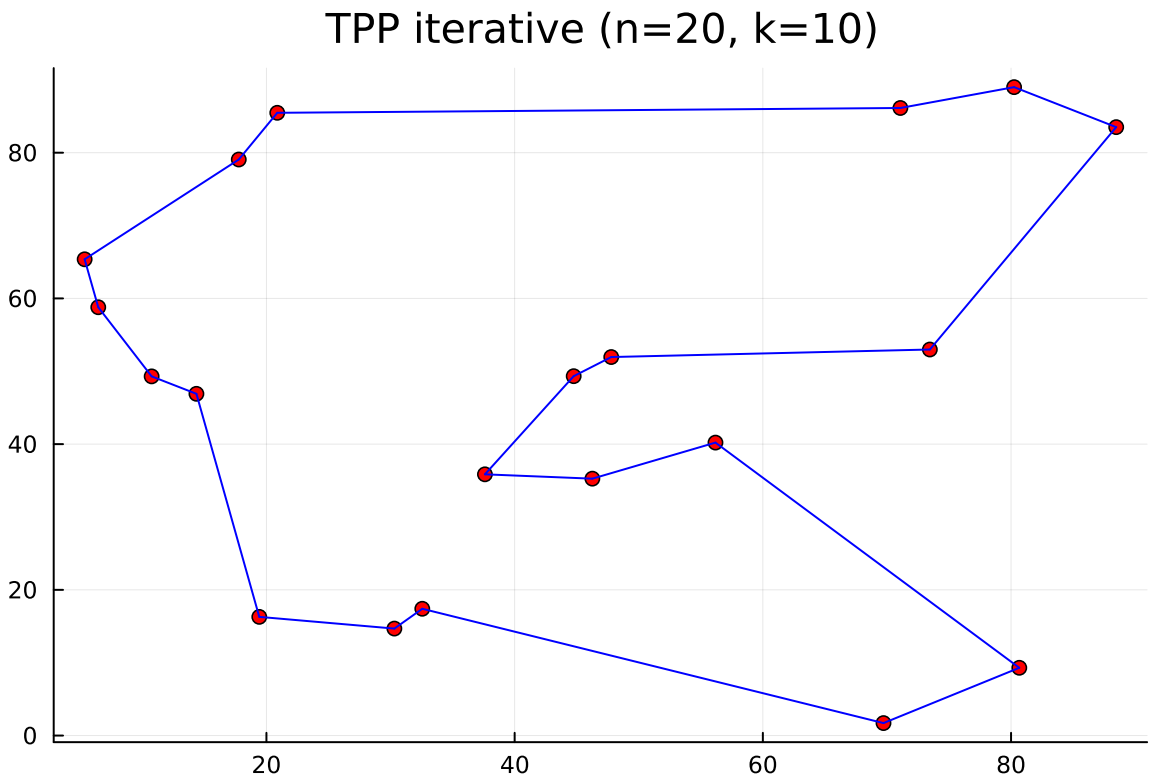
```
     6      30.31      14.68
     7      44.76      49.33
     8      14.36      46.91
     9      20.87      85.48
    10      46.26      35.26
  ... (10 więcej)

Ceny przedmiot × miasto (pierwsze 10 × 6):
               c1      c2      c3      c4      c5      c6
  it 1    139.41  129.40   88.50  147.78   98.50  140.20
  it 2     83.66   73.20   60.94  149.34  118.42   69.29
  it 3    125.20  136.49  140.02  140.99   71.57  133.30
  it 4     94.67   87.77  120.28  131.52  114.82   84.47
  it 5     53.35  126.04  128.97   75.56  100.05  128.05
  it 6    140.54   98.29  135.52   83.54   92.76   89.26
  it 7    140.13   55.84   80.01  136.53  128.25  121.68
  it 8     83.75   68.11  131.79   89.74  119.93  123.48
  it 9    125.53  122.87   75.30  117.21  128.28  120.85
  it10     63.56  109.68  146.37   83.36  143.06  109.32
  ...
```



```
--- Rozwiązanie iteratywne ---
  Found cycle of length 3
  Found cycle of length 3
  Found cycle of length 3
  Found cycle of length 5
  Found cycle of length 4
Czas: 0.008 s
Koszt całkowity: 909.648
```

```julia
y_val = JuMP.value.(model_iter[:y])
println("Decyzje zakupów:")
for item in 1:size(cost_d, 1)
    city = argmax(y_val[item, :])
    price = cost_d[item, city]
    println("  Przedmiot $item -> miasto $city (cena $(round(price,
     ↪  digits=2)))")
end

plot_tpp_tour(X_d, Y_d, JuMP.value.(model_iter[:x]); title="TPP iterative
 ↪  (n=$n_demo, k=$k_demo)")
```

```
Decyzje zakupów:
  Przedmiot 1 -> miasto 17 (cena 53.57)
  Przedmiot 2 -> miasto 12 (cena 59.13)
  Przedmiot 3 -> miasto 14 (cena 53.8)
  Przedmiot 4 -> miasto 12 (cena 53.34)
  Przedmiot 5 -> miasto 1 (cena 53.35)
  Przedmiot 6 -> miasto 18 (cena 59.87)
  Przedmiot 7 -> miasto 2 (cena 55.84)
  Przedmiot 8 -> miasto 11 (cena 52.67)
  Przedmiot 9 -> miasto 12 (cena 50.41)
  Przedmiot 10 -> miasto 1 (cena 63.56)
```



Algorytm genetyczny dla TPP

Chromosom pozostaje permutacją w reprezentacji "next-city" (perm[i] = miasto odwiedzane bezpośrednio po mieście i). Ponieważ trasa odwiedza wszystkie miasta, optymalna decyzja zakupowa dla każdego przedmiotu (bez wagi) to po prostu miasto z najniższą ceną — można ją wyznaczyć analitycznie. Funkcja kosztu = długość trasy + suma minimalnych cen przedmiotów + kara, jeśli permutacja zawiera podtrasę.

```
import Pkg
Pkg.activate(".")
Pkg.add(name="Evolutionary", version="0.11")
using Evolutionary
```

```
Activating project at
 ↪  `~/Desktop/miao/metody-i-algorytmy-optymalizacji/lab4`
 Resolving package versions...
   Project No packages added to or removed from
    ↪  `~/Desktop/miao/metody-i-algorytmy-optymalizacji/lab4/Project.toml`
```

```
    Manifest No packages added to or removed from
     ↪  `~/Desktop/miao/metody-i-algorytmy-optymalizacji/lab4/Manifest.toml`
```

```
# import Pkg

# let
#     pkg_id = Base.find_package("Evolutionary")
#     if pkg_id === nothing
#         Pkg.add(Pkg.PackageSpec(name="Evolutionary", version="0.11"))
#     else
#         info = Pkg.dependencies()
#         for (uuid, dep) in info
#             if dep.name == "Evolutionary"
#                 if dep.version === nothing || dep.version < v"0.11"
#                     Pkg.add(Pkg.PackageSpec(name="Evolutionary",
 ↪  version="0.11"))
#                 end
#                 break
#             end
#         end
#     end
# end

# using Evolutionary
using Random
using Printf

function perm_to_inv(perm::Vector)
    N = length(perm)
    inv = zeros(Int, N)
    for i in 1:N
        m = 1
        while perm[m] != i
            if perm[m] > i
                inv[i] += 1
            end
            m += 1
        end
    end
    return inv
end

function inv_to_perm(inv::Vector)
```

```
    N = length(inv)
    pos = zeros(Int, N)
    for i in N:-1:1
        for m in (i+1):N
            if pos[m] >= inv[i] + 1
                pos[m] += 1
            end
        end
        pos[i] = inv[i] + 1
    end
    perm = zeros(Int, N)
    for i in 1:N
        perm[pos[i]] = i
    end
    return perm
end

function has_subtour(perm::Vector)
    c = 1
    for _ in 1:(length(perm)-1)
        c = perm[c]
        if c == 1
            return true
        end
    end
    return false
end

function mutate_swap(recombinant::AbstractVector;
    rng::AbstractRNG=Random.default_rng())
    N = length(recombinant)
    a = true
    while a || has_subtour(recombinant)
        i = rand(rng, 1:N)
        j = rand(rng, 1:N)
        v = recombinant[i]
        recombinant[i] = recombinant[j]
        recombinant[j] = v
        a = false
    end
    return recombinant
end
```

```julia
# Krzyżowanie oparte o inwersje permutacji
function recombine_inv(v1::T, v2::T;
    rng::AbstractRNG=Random.default_rng()) where {T<:AbstractVector}
    N = length(v1)
    i1 = perm_to_inv(v1)
    i2 = perm_to_inv(v2)
    cp = rand(rng, 2:(N-1))
    i1c = vcat(i1[1:cp], i2[(cp+1):N])
    i2c = vcat(i2[1:cp], i1[(cp+1):N])
    return inv_to_perm(i1c), inv_to_perm(i2c)
end
```

```
recombine_inv (generic function with 1 method)
```

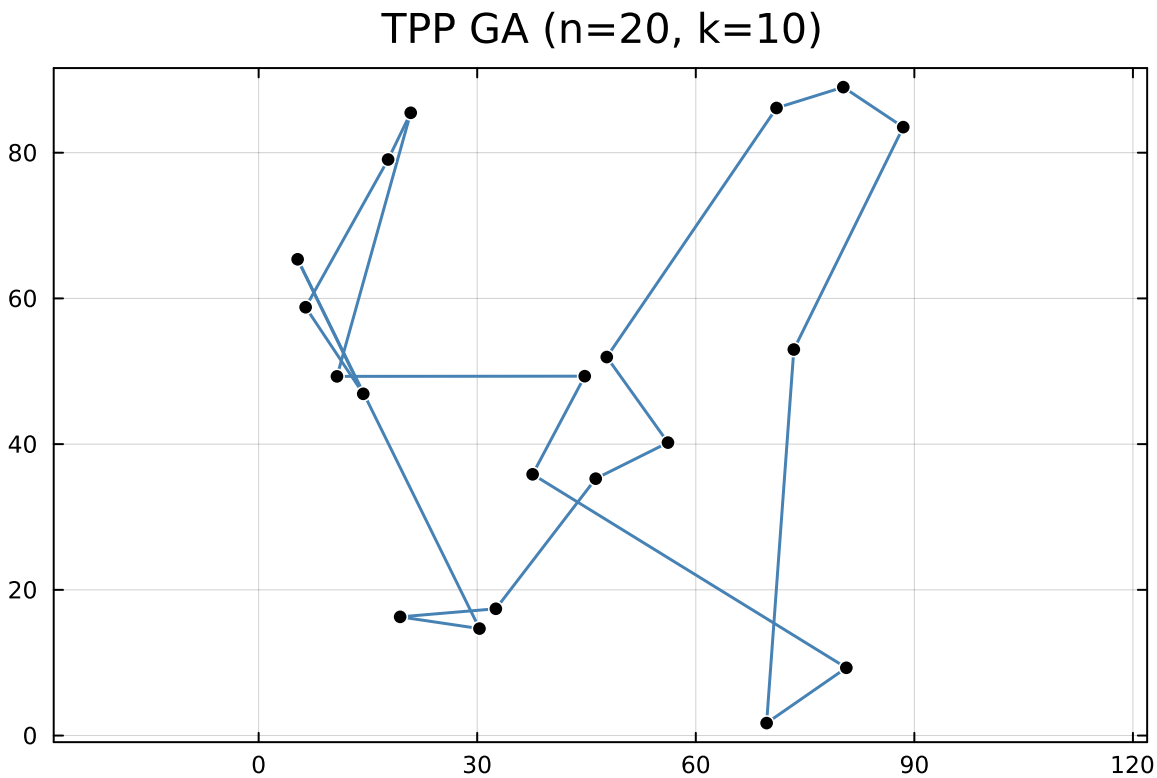
```julia
function tour_order(perm::Vector)
    n = length(perm)
    order = zeros(Int, n)
    order[1] = 1
    for i in 2:n
        order[i] = perm[order[i-1]]
    end
    return order
end

function make_cost_tpp(d, cost_item_city)
    n = size(d, 1)
    k = size(cost_item_city, 1)
    purchase_baseline = 0.0
    for i in 1:k
        purchase_baseline += minimum(@view cost_item_city[i, :])
    end
    function cost(perm::Vector)
        travel = 0.0
        for i in 1:n
            travel += d[i, perm[i]]
        end
        if has_subtour(perm)
            return travel + purchase_baseline + 50000.0
        end
        return travel + purchase_baseline
    end
```

```julia
    return cost
end

function plot_ga_tour(X, Y, perm::Vector; title="")
    plt = Plots.plot(; title=title, legend=false, framestyle=:box,
 ↪  aspect_ratio=:equal)
    for i in 1:length(perm)
        j = perm[i]
        Plots.plot!(plt, [X[i], X[j]], [Y[i], Y[j]]; color=:steelblue,
 ↪  linewidth=1.5)
    end
    Plots.scatter!(plt, X, Y; color=:black, markersize=4,
 ↪  markerstrokecolor=:white)
    return plt
end

# Konwersja rozwiązania JuMP (macierz x) na permutację next-city
function jump_to_perm(x_val::AbstractMatrix)
    n = size(x_val, 1)
    perm = zeros(Int, n)
    visited = falses(n)
    visited[1] = true
    current = 1
    for _ in 1:(n-1)
        next_city = 0
        for j in 1:n
            if x_val[current, j] > 0.5 && !visited[j]
                next_city = j
                break
            end
        end
        @assert next_city != 0
        perm[current] = next_city
        visited[next_city] = true
        current = next_city
    end
    perm[current] = 1
    return perm
end
```

```
jump_to_perm (generic function with 1 method)
```

```julia
x0_demo = [mod1(i + 1, n_demo) for i in 1:n_demo]
cost_fn_demo = make_cost_tpp(d_d, cost_d)
println("Koszt początkowy: $(round(cost_fn_demo(x0_demo), digits=3))")

ga_demo = GA(; populationSize=500, crossover=recombine_inv,
    mutation=mutate_swap, epsilon=0.05, crossoverRate=0.9)
opts_demo = Evolutionary.Options(; iterations=500, time_limit=30.0)

t_ga_demo = @elapsed (res_ga_demo = Evolutionary.optimize(cost_fn_demo,
 ↪  x0_demo, ga_demo, opts_demo))
println("Koszt GA: $(round(res_ga_demo.minimum, digits=3))")
println("Czas GA: $(round(t_ga_demo, digits=3)) s")
println("Podtrasa? $(has_subtour(res_ga_demo.minimizer))")

plot_ga_tour(X_d, Y_d, res_ga_demo.minimizer; title="TPP GA (n=$n_demo,
 ↪  k=$k_demo)")
```

```
Koszt początkowy: 1506.426
Koszt GA: 1046.428
Czas GA: 0.929 s
Podtrasa? false
```



Porównanie 9 losowych problemów

Po trzy problemy dla każdego rozmiaru (n=50, k=20), (n=50, k=50) oraz (n=100, k=100).

```
function ga_params_for(n)
    if n <= 50
        return GA(; populationSize=500, crossover=recombine_inv,
            mutation=mutate_swap, epsilon=0.05, crossoverRate=0.9),
        Evolutionary.Options(; iterations=2000, time_limit=55.0)
    else
        return GA(; populationSize=300, crossover=recombine_inv,
            mutation=mutate_swap, epsilon=0.05, crossoverRate=0.9),
        Evolutionary.Options(; iterations=2000, time_limit=55.0)
    end
end

problems_30 = [(50, 20, 1), (50, 20, 2), (50, 20, 3),
    (50, 50, 4), (50, 50, 5), (50, 50, 6),
    (100, 100, 7), (100, 100, 8), (100, 100, 9)]
```

```julia
results_30 = Any[]
for (np, kp, seed) in problems_30
    println("Problem n=$np k=$kp seed=$seed")
    Xp, Yp, dp, cp = generate_random_tpp(np, kp; random_seed=seed)

    model_p, t_jump = solve_tpp_iterative(dp, cp)
    cost_jump = objective_value(model_p)

    cost_fn_p = make_cost_tpp(dp, cp)
    x0_p = [mod1(i + 1, np) for i in 1:np]
    ga_p, opts_p = ga_params_for(np)
    t_ga = @elapsed (res_ga = Evolutionary.optimize(cost_fn_p, x0_p, ga_p,
 ↪  opts_p))
    cost_ga = res_ga.minimum

    push!(results_30, (n=np, k=kp, seed=seed,
        jump_cost=cost_jump, jump_time=t_jump,
        ga_cost=cost_ga, ga_time=t_ga))
    @printf "  JuMP: koszt=%.2f czas=%.2fs    GA: koszt=%.2f czas=%.2fs\n"
     ↪  cost_jump t_jump cost_ga t_ga
end

println("\n=== Tabela porównawcza (3.0) ===")
@printf "%4s %4s %5s %12s %10s %12s %10s %8s\n" "n" "k" "seed" "JuMP koszt"
 ↪  "JuMP t[s]" "GA koszt" "GA t[s]" "GA/JuMP"
for r in results_30
    @printf "%4d %4d %5d %12.2f %10.2f %12.2f %10.2f %8.3f\n" r.n r.k r.seed
     ↪  r.jump_cost r.jump_time r.ga_cost r.ga_time (r.ga_cost / r.jump_cost)
end
```

```
Problem n=50 k=20 seed=1
  JuMP: koszt=1569.95 czas=0.02s    GA: koszt=2369.41 czas=0.20s
Problem n=50 k=20 seed=2
  JuMP: koszt=1612.07 czas=0.72s    GA: koszt=2447.88 czas=0.11s
Problem n=50 k=20 seed=3
  JuMP: koszt=1679.49 czas=0.08s    GA: koszt=2473.55 czas=0.18s
Problem n=50 k=50 seed=4
  JuMP: koszt=3239.97 czas=0.37s    GA: koszt=3803.32 czas=0.26s
Problem n=50 k=50 seed=5
  JuMP: koszt=3149.27 czas=2.90s    GA: koszt=3794.74 czas=0.21s
```

```
Problem n=50 k=50 seed=6
  JuMP: koszt=3190.33 czas=3.08s    GA: koszt=4193.70 czas=0.11s
Problem n=100 k=100 seed=7
  JuMP: koszt=5860.38 czas=322.44s    GA: koszt=8084.94 czas=0.58s
Problem n=100 k=100 seed=8
  JuMP: koszt=5890.58 czas=12.40s    GA: koszt=8102.08 czas=0.58s
Problem n=100 k=100 seed=9
  JuMP: koszt=5899.56 czas=18.80s    GA: koszt=9126.61 czas=0.17s

=== Tabela porównawcza (3.0) ===
```

n	k	seed	JuMP koszt	JuMP t[s]	GA koszt	GA t[s]	GA/JuMP
50	20	1	1569.95	0.02	2369.41	0.20	1.509
50	20	2	1612.07	0.72	2447.88	0.11	1.518
50	20	3	1679.49	0.08	2473.55	0.18	1.473
50	50	4	3239.97	0.37	3803.32	0.26	1.174
50	50	5	3149.27	2.90	3794.74	0.21	1.205
50	50	6	3190.33	3.08	4193.70	0.11	1.315
100	100	7	5860.38	322.44	8084.94	0.58	1.380
100	100	8	5890.58	12.40	8102.08	0.58	1.375
100	100	9	5899.56	18.80	9126.61	0.17	1.547

Na 4.0

Wagi przedmiotów

Dodano do problemu wagę przedmiotów. Koszt rozwiązania powiększa się o `alpha * weight[i] * remaining_dist[i]`, gdzie `remaining_dist[i]` to długość trasy od miasta zakupu przedmiotu `i` do końca trasy (powrót do miasta startowego). Współczynnik `alpha` dobrano tak, aby dla przedmiotu o wadze średniej transport między najbardziej odległymi miastami stanowił ~1% średniej ceny:

```
alpha = 0.01 * mean(cost_item_city) / (mean(weights) * maximum(d))
```

Składnik wagowy uwzględniamy tylko w funkcji kosztu algorytmu genetycznego — model JuMP rozwiązuje wersję bez wag (używamy go jako baseline).

```julia
function generate_weights(k; random_seed=1)
    rng = Random.MersenneTwister(random_seed)
    return 0.5 .+ rand(rng, k)  # w   [0.5, 1.5], średnia   1.0
end

function compute_alpha(d, cost_item_city, weights)
    avg_weight = sum(weights) / length(weights)
```

```
    max_distance = maximum(d)
    avg_price = sum(cost_item_city) / length(cost_item_city)
    return 0.01 * avg_price / (avg_weight * max_distance)
end

function make_cost_tpp_weighted(d, cost_item_city, weights, alpha)
    n = size(d, 1)
    k = size(cost_item_city, 1)
    function cost(perm::Vector)
        travel = 0.0
        for i in 1:n
            travel += d[i, perm[i]]
        end
        if has_subtour(perm)
            return travel + 1e8
        end
        # Trasa zaczyna i kończy się w mieście 1
        order = zeros(Int, n)
        order[1] = 1
        for i in 2:n
            order[i] = perm[order[i-1]]
        end
        position = zeros(Int, n)
        for i in 1:n
            position[order[i]] = i
        end
        # remaining_dist[p] = suma długości krawędzi od pozycji p do końca
 ↪  trasy (pozycji n)
        remaining_dist = zeros(n)
        for p in (n-1):-1:1
            remaining_dist[p] = remaining_dist[p+1] + d[order[p], order[p+1]]
        end
        purchase = 0.0
        for item in 1:k
            best = Inf
            for c in 1:n
                tot = cost_item_city[item, c] + alpha * weights[item] *
↪   remaining_dist[position[c]]
                if tot < best
                    best = tot
                end
            end
```

```
            purchase += best
        end
        return travel + purchase
    end
    return cost
end
```

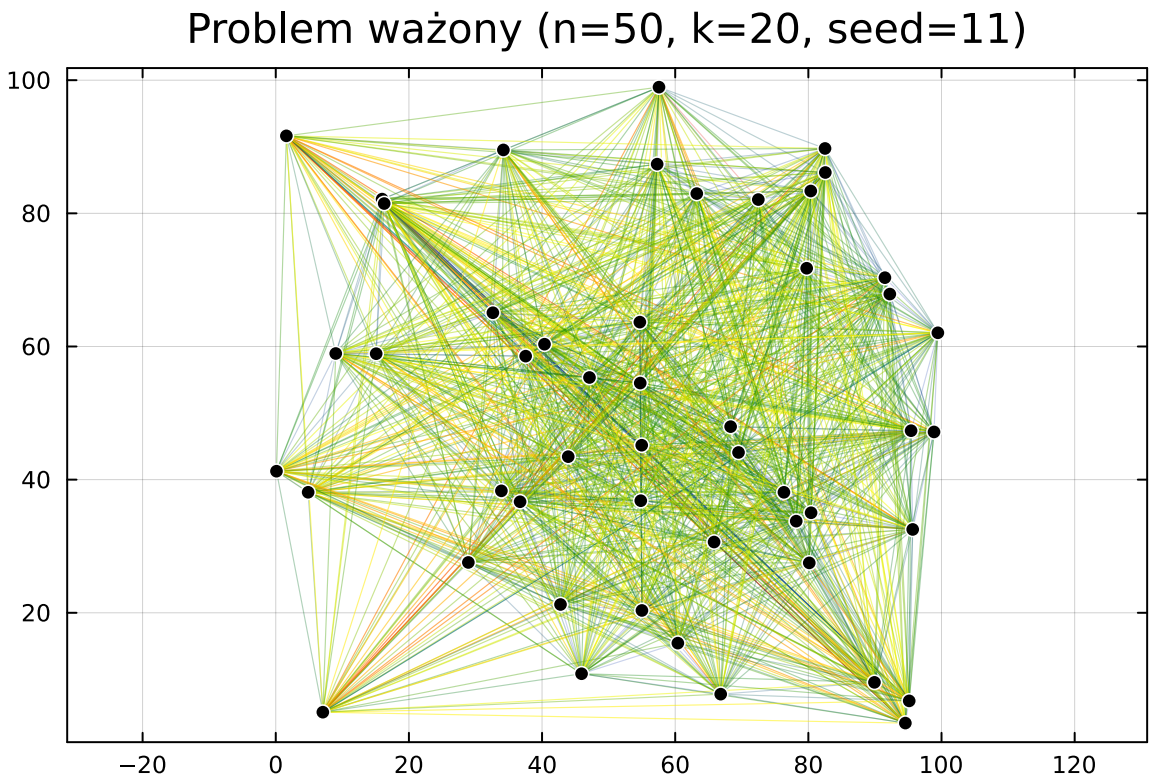
```
make_cost_tpp_weighted (generic function with 1 method)
```

```
println("=== Porównanie GA z wagą i bez (oraz JuMP bez wagi) ===")
results_weights = Any[]
for (np, kp, seed) in [(50, 20, 11), (50, 50, 12), (100, 100, 13)]
    Xp, Yp, dp, cp = generate_random_tpp(np, kp; random_seed=seed)
    weights = generate_weights(kp; random_seed=seed)
    alpha = compute_alpha(dp, cp, weights)
    println("\nProblem n=$np k=$kp seed=$seed alpha=$(round(alpha,
     ↪  sigdigits=4))")
    print_problem_info(Xp, Yp, dp, cp; weights=weights)
    display(plot_problem(Xp, Yp, dp; title="Problem ważony (n=$np, k=$kp,
     ↪  seed=$seed)"))

    # JuMP (bez wagi)
    model_p, t_jump = solve_tpp_iterative(dp, cp)
    perm_jump = jump_to_perm(JuMP.value.(model_p[:x]))

    # GA bez wagi
    cost_unw = make_cost_tpp(dp, cp)
    x0_p = [mod1(i + 1, np) for i in 1:np]
    ga_p, opts_p = ga_params_for(np)
    t_ga_unw = @elapsed (res_ga_unw = Evolutionary.optimize(cost_unw, x0_p,
 ↪  ga_p, opts_p))

    # GA z wagą
    cost_w = make_cost_tpp_weighted(dp, cp, weights, alpha)
    t_ga_w = @elapsed (res_ga_w = Evolutionary.optimize(cost_w, x0_p, ga_p,
 ↪  opts_p))

    # Ewaluacja każdego rozwiązania pod oboma kryteriami
    j_unw = cost_unw(perm_jump)
    j_w = cost_w(perm_jump)
    gu_unw = cost_unw(res_ga_unw.minimizer)
    gu_w = cost_w(res_ga_unw.minimizer)
```

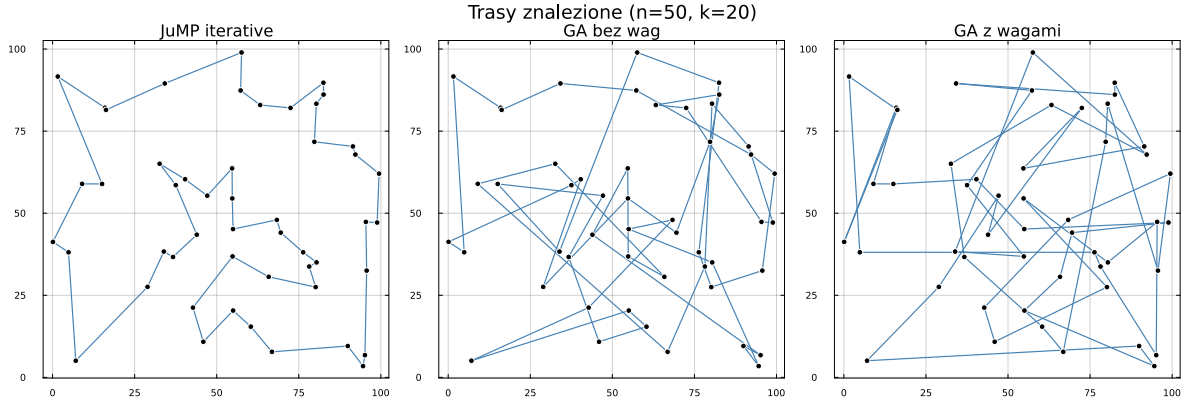
```
    gw_unw = cost_unw(res_ga_w.minimizer)
    gw_w = cost_w(res_ga_w.minimizer)

    push!(results_weights, (n=np, k=kp,
        j_unw=j_unw, j_w=j_w,
        gu_unw=gu_unw, gu_w=gu_w,
        gw_unw=gw_unw, gw_w=gw_w,
        t_jump=t_jump, t_ga_unw=t_ga_unw, t_ga_w=t_ga_w))

    @printf "  JuMP (bez wag):    bez_wag=%.2f  z_wag=%.2f\n" j_unw j_w
    @printf "  GA (bez wag):      bez_wag=%.2f  z_wag=%.2f\n" gu_unw gu_w
    @printf "  GA (z wagami):     bez_wag=%.2f  z_wag=%.2f\n" gw_unw gw_w

    display(Plots.plot(
        plot_ga_tour(Xp, Yp, perm_jump; title="JuMP iterative"),
        plot_ga_tour(Xp, Yp, res_ga_unw.minimizer; title="GA bez wag"),
        plot_ga_tour(Xp, Yp, res_ga_w.minimizer; title="GA z wagami");
        layout=(1, 3), size=(1500, 500),
        plot_title="Trasy znalezione (n=$np, k=$kp)"
    ))
end

println("\n=== Tabela (4.0 - wagi) ===")
@printf "%4s %4s | %10s %10s | %10s %10s | %10s %10s\n" "n" "k" "JuMP bw"
 ↪  "JuMP zw" "GAbw bw" "GAbw zw" "GAzw bw" "GAzw zw"
for r in results_weights
    @printf "%4d %4d | %10.2f %10.2f | %10.2f %10.2f | %10.2f %10.2f\n" r.n
     ↪  r.k r.j_unw r.j_w r.gu_unw r.gu_w r.gw_unw r.gw_w
end
println("bw = funkcja celu bez wag, zw = z wagami; GAbw = GA optymalizujące
 ↪  bez wag, GAzw = GA z wagami.")
```

```
=== Porównanie GA z wagą i bez (oraz JuMP bez wagi) ===

Problem n=50 k=20 seed=11 alpha=0.007604
Liczba miast n = 50,  liczba przedmiotów k = 20
Odległości:    min=0.73   średnia=50.10   max=128.16
Ceny:          min=50.36   średnia=99.73   max=149.98

Współrzędne miast (pierwsze 10):
    id          X          Y
     1      33.89      38.32
```

```
     2      57.57      98.96
     3      92.24      67.87
     4      40.37      60.34
     5      54.97      45.17
     6      15.97      82.14
     7       4.87      38.10
     8       1.60      91.64
     9      98.87      47.16
    10      16.29      81.47
  ... (40 więcej)

Ceny przedmiot × miasto (pierwsze 10 × 6):
              c1      c2      c3      c4      c5      c6
  it 1   137.98   50.71  129.30   56.11   91.82   84.68
  it 2    88.33  126.85  149.66   58.03   71.21  118.55
  it 3    77.09  122.25  125.62  107.73   50.62   81.29
  it 4    82.95   94.51   57.81  125.05   76.98  123.24
  it 5    76.01   66.72  144.47  144.97  122.11  100.59
  it 6    59.84   85.58  125.12  149.90   94.76   59.89
  it 7    78.06  133.71  136.16  144.75  148.68   80.51
  it 8    95.34  125.04  104.39  119.39   84.64   52.94
  it 9    70.56   71.09  116.14  123.18   90.59   72.67
  it10   114.66   59.84  116.28  119.35   66.38   90.71
  ...

Wagi przedmiotów (pierwsze 10):
  item     weight
     1      0.839
     2      1.076
     3      1.422
     4      0.904
     5      1.050
     6      0.660
     7      0.549
     8      0.516
     9      1.489
    10      0.663
  ... (10 więcej)
```

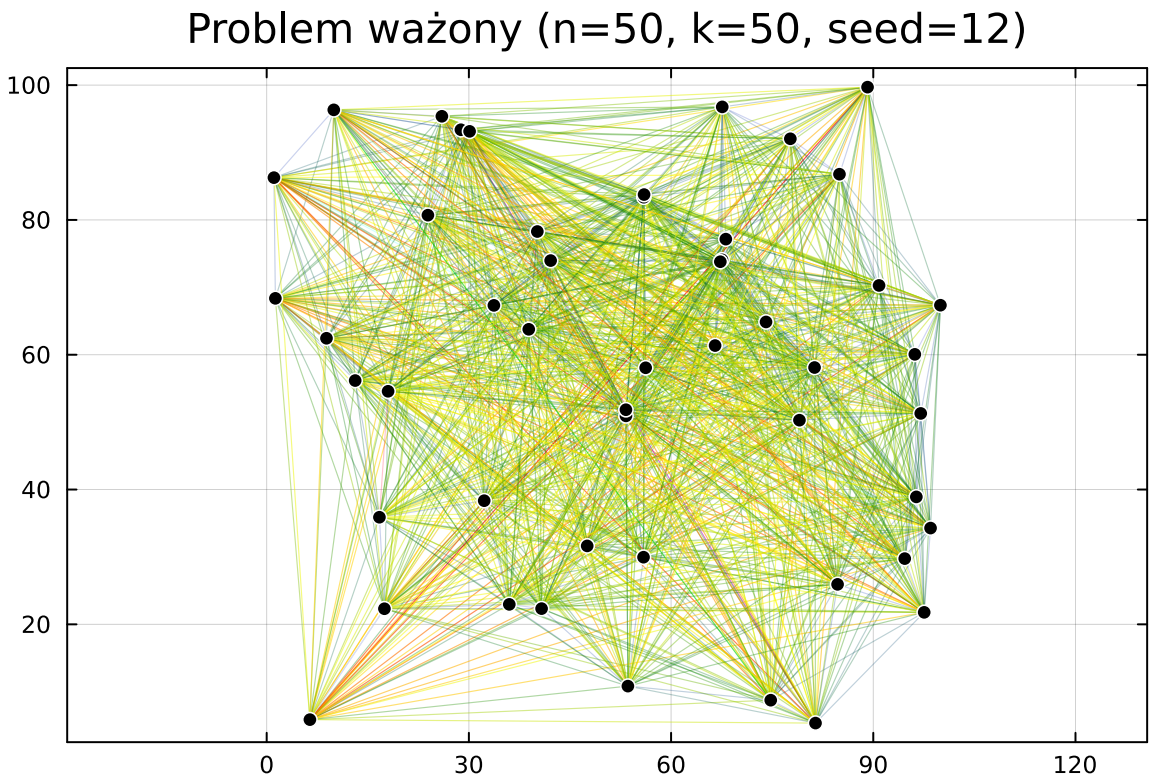



```
JuMP (bez wag):     bez_wag=1652.60  z_wag=1689.38
GA (bez wag):       bez_wag=2420.41  z_wag=2499.82
GA (z wagami):      bez_wag=2662.97  z_wag=2738.70
```

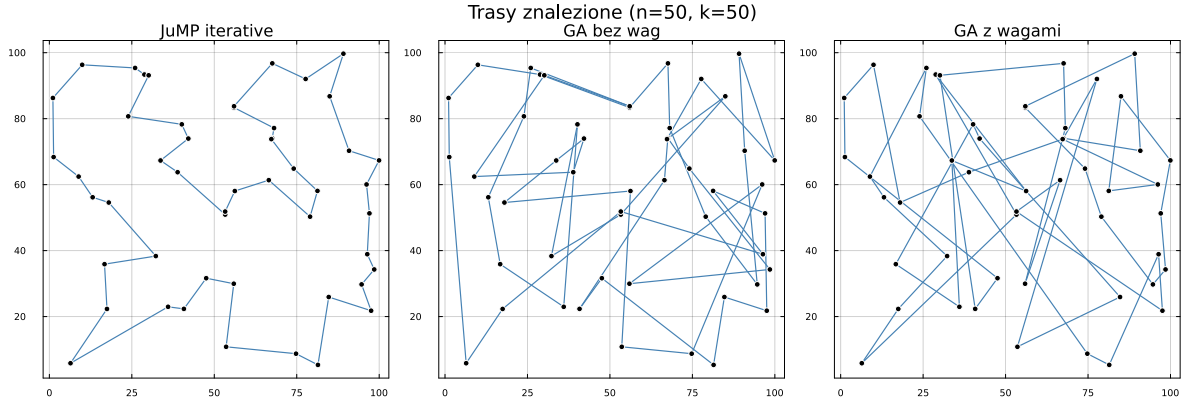



```
Problem n=50 k=50 seed=12 alpha=0.007682
Liczba miast n = 50,  liczba przedmiotów k = 50
Odległości:    min=0.37   średnia=51.79   max=125.10
```

```
Ceny:          min=50.01   średnia=100.59   max=149.96

Współrzędne miast (pierwsze 10):
    id          X          Y
     1      66.53      61.36
     2      55.91      29.96
     3      68.12      77.15
     4      47.55      31.64
     5       1.29      68.37
     6       1.08      86.27
     7      74.09      64.86
     8      55.96      83.37
     9      56.23      58.06
    10      28.81      93.38
  ... (40 więcej)

Ceny przedmiot × miasto (pierwsze 10 × 6):
              c1      c2      c3      c4      c5      c6
  it 1   112.51  118.26  115.87  108.88   94.65   93.39
  it 2   102.19  132.20   80.95  119.23  112.41  126.20
  it 3   133.51  122.73  142.45   76.67  112.89  127.44
  it 4   103.23   89.53   94.20   77.72  148.20  130.54
  it 5   110.81  117.05  112.83  142.48  131.81  126.58
  it 6   143.70   83.52   73.10  138.97   78.62  100.20
  it 7    86.03  131.43   88.45  148.82   62.52   88.60
  it 8   111.65  127.17  141.61   91.43   92.04  149.92
  it 9    54.20  128.70  143.87  113.54  110.87   71.90
  it10    54.23   98.06  143.29   54.89  121.07   69.65
  ...

Wagi przedmiotów (pierwsze 10):
  item     weight
     1      1.165
     2      1.059
     3      1.181
     4      0.976
     5      0.513
     6      0.511
     7      1.241
     8      1.060
     9      1.062
    10      0.788
  ... (40 więcej)
```

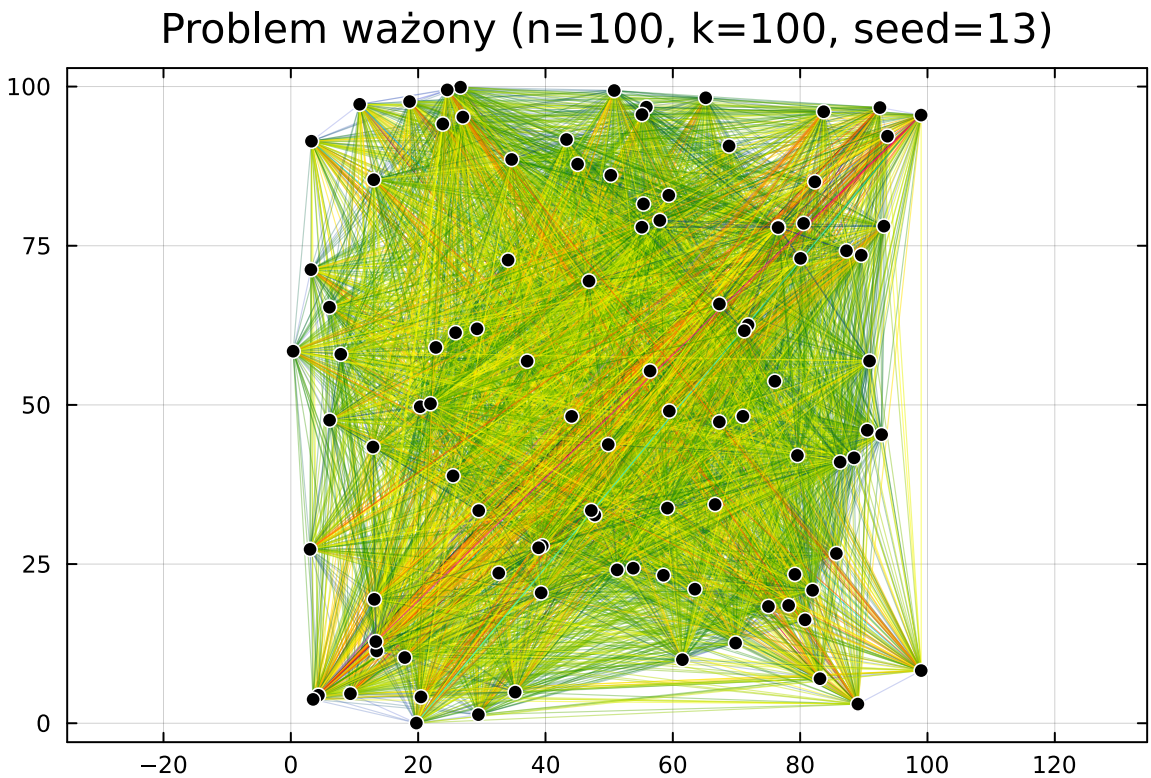



```
JuMP (bez wag):     bez_wag=3171.81  z_wag=3248.77
GA (bez wag):       bez_wag=4073.76  z_wag=4230.67
GA (z wagami):      bez_wag=4159.64  z_wag=4348.58
```

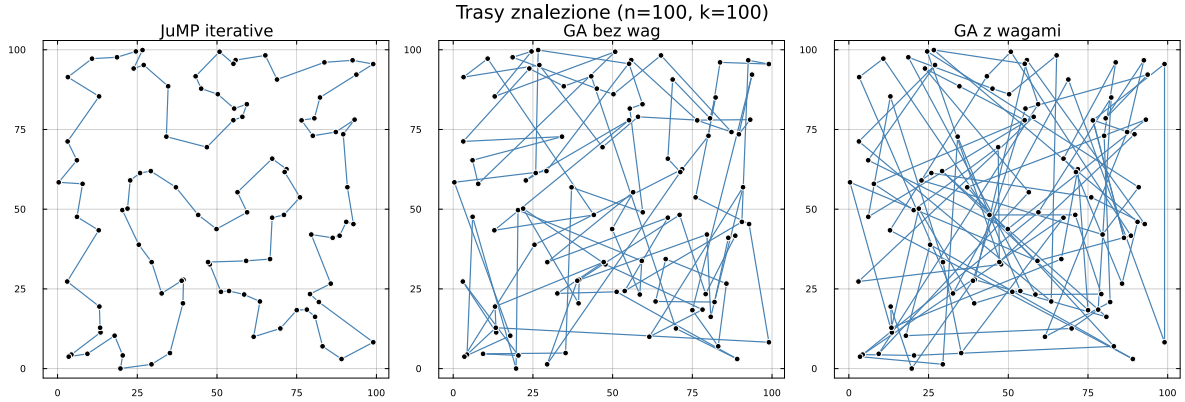



```
Problem n=100 k=100 seed=13 alpha=0.007508
Liczba miast n = 100,  liczba przedmiotów k = 100
Odległości:    min=0.09   średnia=54.19   max=132.45
```

```
Ceny:          min=50.01   średnia=99.81   max=150.00

Współrzędne miast (pierwsze 10):
    id          X          Y
     1      68.83      90.67
     2      32.66      23.58
     3       3.24      91.42
     4      10.81      97.22
     5      44.11      48.20
     6      71.01      48.21
     7      79.18      23.38
     8      58.52      23.24
     9      18.65      97.65
    10      45.05      87.81
  ... (90 więcej)

Ceny przedmiot × miasto (pierwsze 10 × 6):
              c1      c2      c3      c4      c5      c6
  it 1   128.79   79.23  113.18   53.69   94.53  125.15
  it 2    95.40   71.05  141.01  129.67  115.19   85.34
  it 3    59.92  125.42   88.77  138.01  128.38   66.62
  it 4    97.40   68.05   70.15  119.50   96.13   83.48
  it 5    50.67  108.57  108.05  132.55  109.36   94.49
  it 6   146.99  106.52   96.70   51.31   54.41   59.25
  it 7   104.81  128.65  147.30  121.94   70.91   69.36
  it 8    55.84   71.52   67.10  122.31  100.09   55.27
  it 9   147.02   91.58   57.75  135.76   86.18  109.32
  it10    74.83  121.10  140.52  115.36   86.59   84.39
  ...

Wagi przedmiotów (pierwsze 10):
  item     weight
     1      1.188
     2      0.827
     3      0.532
     4      0.608
     5      0.941
     6      1.210
     7      1.292
     8      1.085
     9      0.686
    10      0.950
  ... (90 więcej)
```




```
JuMP (bez wag):     bez_wag=5862.24  z_wag=6061.96
GA (bez wag):       bez_wag=7905.44  z_wag=8406.28
GA (z wagami):      bez_wag=9518.42  z_wag=10012.44
```




```
=== Tabela (4.0 - wagi) ===
   n     k |     JuMP bw     JuMP zw |     GAbw bw     GAbw zw |     GAzw bw
    ↪  GAzw zw
```

```
  50   20 |    1652.60    1689.38 |    2420.41    2499.82 |    2662.97
  ↪  2738.70
  50   50 |    3171.81    3248.77 |    4073.76    4230.67 |    4159.64
  ↪  4348.58
 100  100 |    5862.24    6061.96 |    7905.44    8406.28 |    9518.42
  ↪  10012.44
bw = funkcja celu bez wag, zw = z wagami; GAbw = GA optymalizujące bez wag,
 ↪  GAzw = GA z wagami.
```

Krzyżowanie PMX

PMX stosujemy bezpośrednio na permutacji next-city — wynik pozostaje permutacją zbioru `{1..n}`, choć może zawierać podtrasy (są karane przez funkcję kosztu, jak w przypadku krzyżowania przez inwersję).

```julia
using Random
```

```julia
function pmx_crossover(p1::AbstractVector, p2::AbstractVector, cx1::Int,
 ↪  cx2::Int)
    n = length(p1)
    child = zeros(Int, n)
    @inbounds for q in cx1:cx2
        child[q] = p1[q]
    end
    seg1 = Set(@view p1[cx1:cx2])
    @inbounds for i in cx1:cx2
        v = p2[i]
        if !(v in seg1)
            j = i
            while child[j] != 0
                w = child[j]
                jnew = 0
                for q in 1:n
                    if p2[q] == w
                        jnew = q
                        break
                    end
                end
                j = jnew
            end
            child[j] = v
```

```julia
        end
    end
    @inbounds for i in 1:n
        if child[i] == 0
            child[i] = p2[i]
        end
    end
    return child
end

function recombine_pmx(v1::T, v2::T;
    rng::AbstractRNG=Random.default_rng()) where {T<:AbstractVector}
    N = length(v1)
    a, b = rand(rng, 1:N), rand(rng, 1:N)
    cx1, cx2 = min(a, b), max(a, b)
    if cx1 == cx2
        return copy(v1), copy(v2)
    end
    return pmx_crossover(v1, v2, cx1, cx2), pmx_crossover(v2, v1, cx1, cx2)
end

# test PMX
let
    p1 = [3, 4, 8, 2, 7, 1, 6, 5]
    p2 = [4, 2, 5, 1, 6, 8, 3, 7]
    c1 = pmx_crossover(p1, p2, 4, 6)
    c2 = pmx_crossover(p2, p1, 4, 6)
    @assert sort(c1) == 1:8
    @assert sort(c2) == 1:8
    println("PMX dzieci: ", c1, " ", c2)
end
```

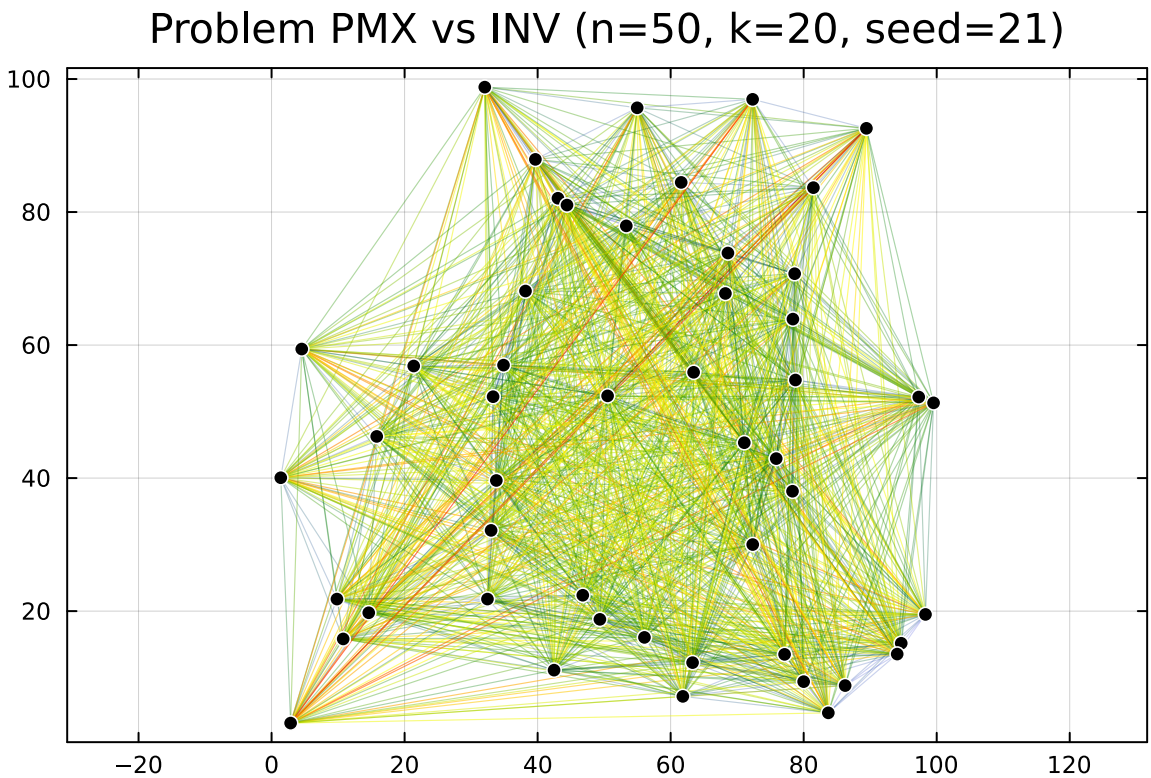
```
PMX dzieci: [4, 8, 5, 2, 7, 1, 3, 6] [3, 4, 2, 1, 6, 8, 7, 5]
```

```julia
println("=== Porównanie PMX vs inwersja ===")
results_pmx = Any[]
for (np, kp, seed) in [(50, 20, 21), (50, 50, 22), (100, 100, 23)]
    Xp, Yp, dp, cp = generate_random_tpp(np, kp; random_seed=seed)
    println("\n=== Problem n=$np k=$kp seed=$seed ===")
    print_problem_info(Xp, Yp, dp, cp)
    display(plot_problem(Xp, Yp, dp; title="Problem PMX vs INV (n=$np, k=$kp,
     ↪  seed=$seed)"))
```

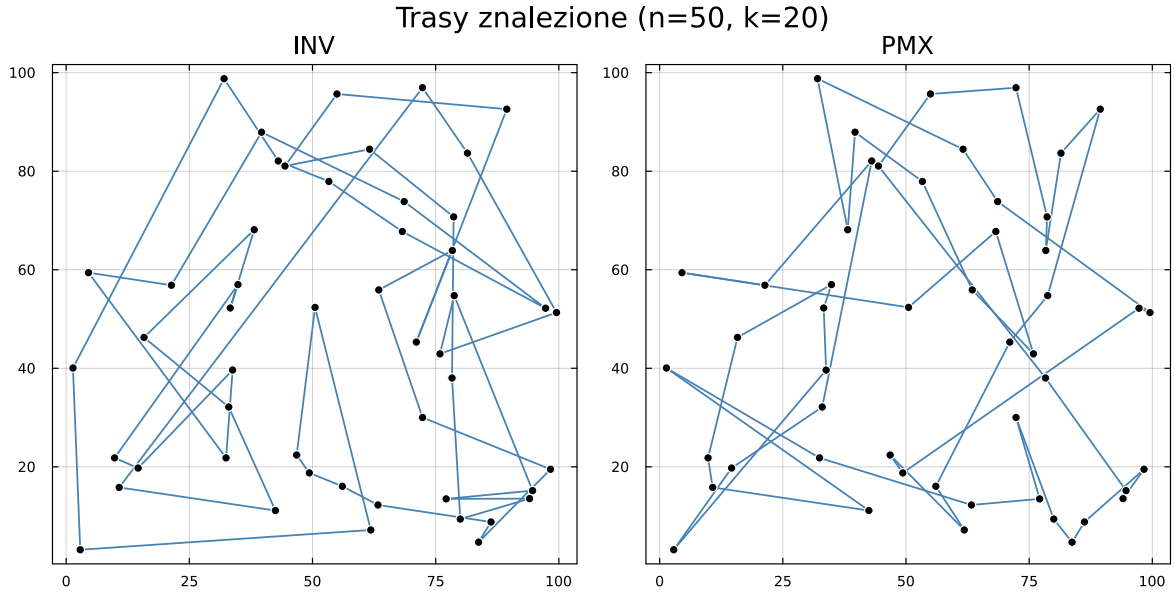
```
    cost_fn_p = make_cost_tpp(dp, cp)
    x0_p = [mod1(i + 1, np) for i in 1:np]
    pop = (np <= 50 ? 500 : 300)
    opts = Evolutionary.Options(; iterations=2000, time_limit=55.0)

    ga_inv = GA(; populationSize=pop, crossover=recombine_inv,
        mutation=mutate_swap, epsilon=0.05, crossoverRate=0.9)
    t_inv = @elapsed (res_inv = Evolutionary.optimize(cost_fn_p, x0_p,
 ↪  ga_inv, opts))

    ga_pmx = GA(; populationSize=pop, crossover=recombine_pmx,
        mutation=mutate_swap, epsilon=0.05, crossoverRate=0.9)
    t_pmx = @elapsed (res_pmx = Evolutionary.optimize(cost_fn_p, x0_p,
 ↪  ga_pmx, opts))

    push!(results_pmx, (n=np, k=kp, seed=seed,
        inv_cost=res_inv.minimum, inv_time=t_inv,
        pmx_cost=res_pmx.minimum, pmx_time=t_pmx))

    @printf "n=%d k=%d  INV: koszt=%.2f t=%.2fs   PMX: koszt=%.2f t=%.2fs\n"
     ↪  np kp res_inv.minimum t_inv res_pmx.minimum t_pmx

    display(Plots.plot(
        plot_ga_tour(Xp, Yp, res_inv.minimizer; title="INV"),
        plot_ga_tour(Xp, Yp, res_pmx.minimizer; title="PMX");
        layout=(1, 2), size=(1000, 500),
        plot_title="Trasy znalezione (n=$np, k=$kp)"
    ))
end

println("\n=== Tabela (4.0 - PMX vs INV) ===")
@printf "%4s %4s %5s | %12s %10s | %12s %10s | %8s\n" "n" "k" "seed" "INV
 ↪  koszt" "INV t[s]" "PMX koszt" "PMX t[s]" "PMX/INV"
for r in results_pmx
    @printf "%4d %4d %5d | %12.2f %10.2f | %12.2f %10.2f | %8.3f\n" r.n r.k
     ↪  r.seed r.inv_cost r.inv_time r.pmx_cost r.pmx_time (r.pmx_cost /
     ↪  r.inv_cost)
end
```

```
=== Porównanie PMX vs inwersja ===

=== Problem n=50 k=20 seed=21 ===
```

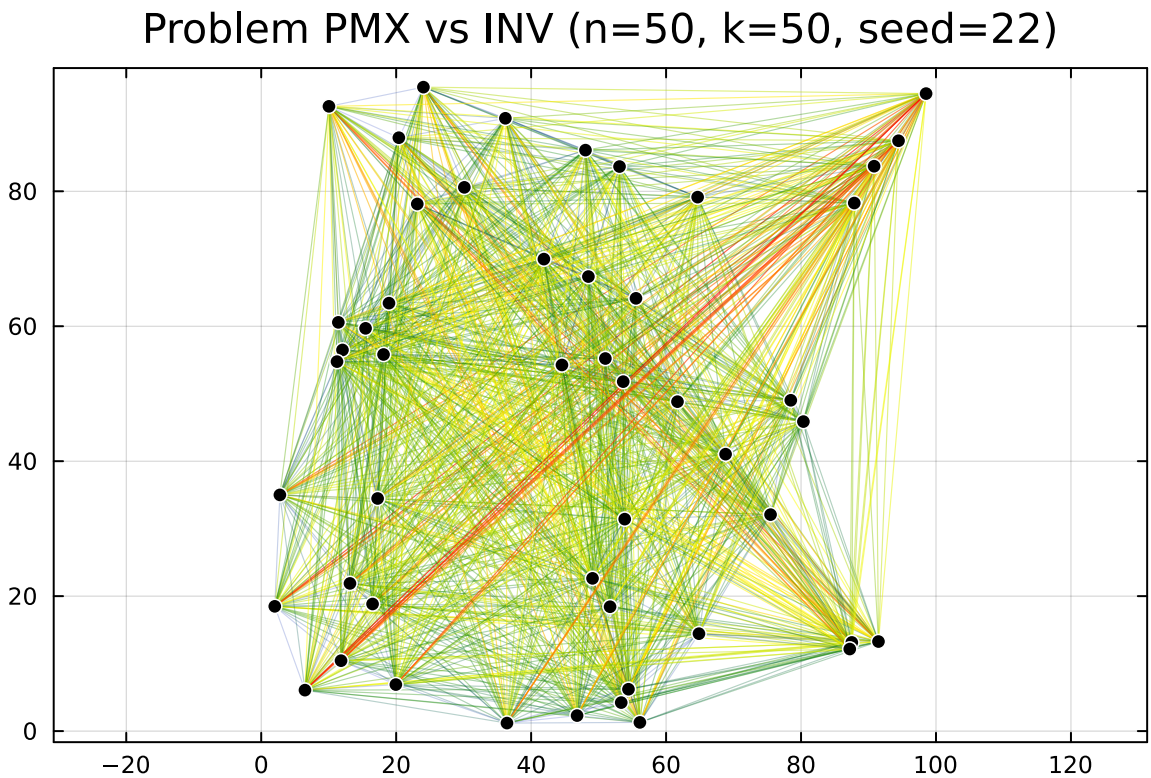
```
Liczba miast n = 50,  liczba przedmiotów k = 20
Odległości:    min=1.71   średnia=51.53   max=124.46
Ceny:          min=50.04   średnia=99.12   max=149.96

Współrzędne miast (pierwsze 10):
    id          X          Y
     1      94.67      15.17
     2      61.59      84.46
     3      10.77      15.82
     4      53.33      77.92
     5      68.23      67.76
     6      33.30      52.24
     7      78.32      38.02
     8      98.31      19.50
     9      33.01      32.14
    10      97.30      52.19
  ... (40 więcej)

Ceny przedmiot × miasto (pierwsze 10 × 6):
              c1      c2      c3      c4      c5      c6
  it 1   106.55   84.07   87.49   76.46   60.35  142.60
  it 2   143.99   55.52  126.54  104.73  137.26   55.72
  it 3   145.60   58.88  115.95   53.64  101.71  113.70
  it 4    62.90  114.22  108.02  119.68  140.30   95.38
  it 5    59.88   59.82   71.35  138.45   54.94  141.27
  it 6    62.59  111.56   68.58   70.72   53.63   55.38
  it 7    71.80   77.12   77.19   76.60   84.77  117.44
  it 8   118.45   58.65   82.90  108.26  130.12   65.44
  it 9   134.41  126.70  110.62  137.59   51.88   53.72
  it10    50.32  113.63  129.10  108.89  131.29  110.87
  ...
```



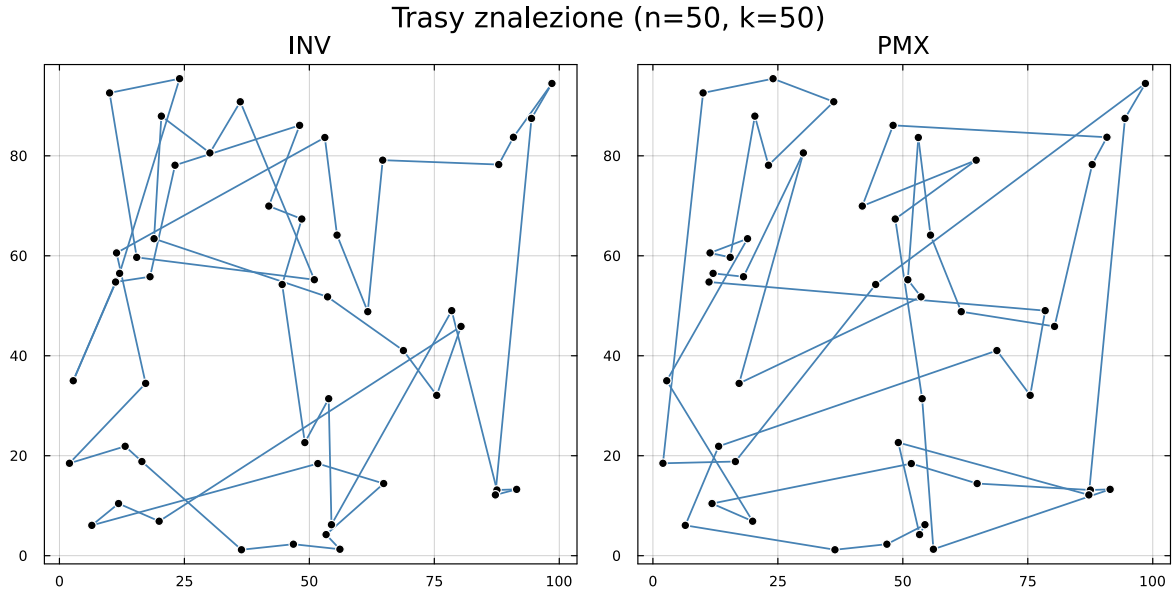

```
n=50 k=20  INV: koszt=2396.34 t=0.21s   PMX: koszt=2191.45 t=0.31s
```



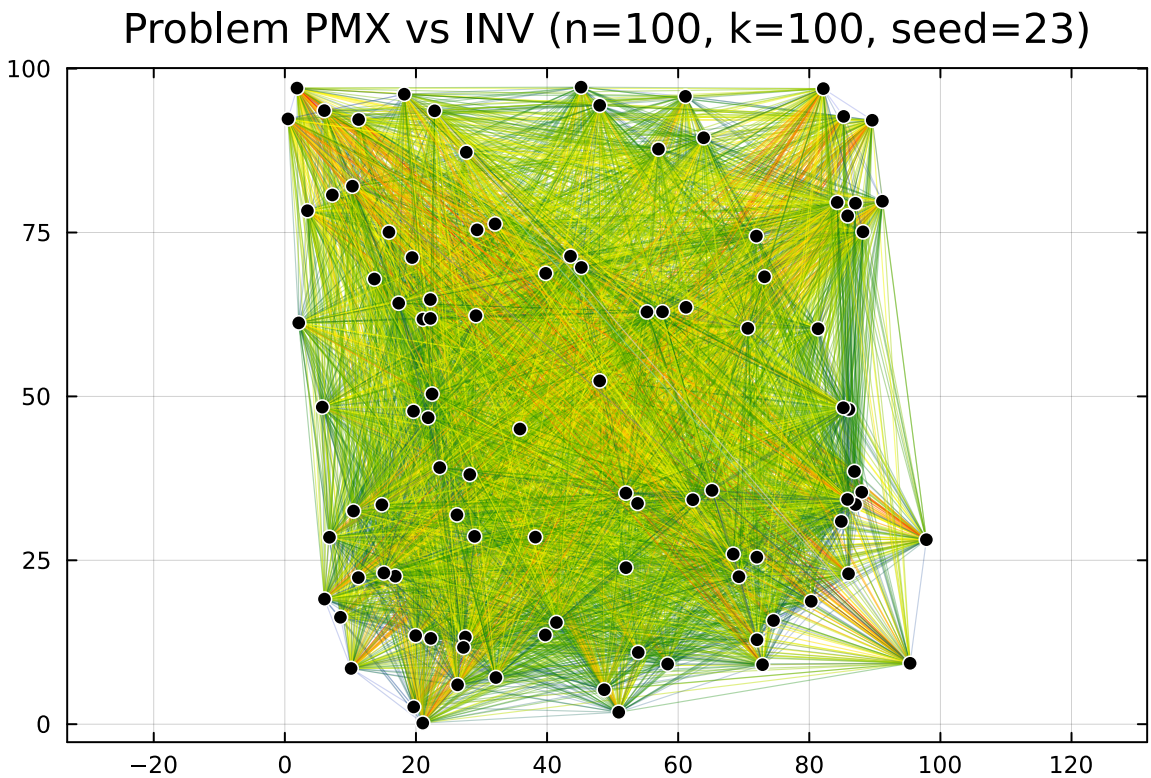
```
=== Problem n=50 k=50 seed=22 ===
Liczba miast n = 50,  liczba przedmiotów k = 50
Odległości:    min=1.02   średnia=53.22   max=127.63
Ceny:          min=50.04   średnia=100.31   max=149.99

Współrzędne miast (pierwsze 10):
    id          X          Y
     1      30.10      80.58
     2      16.50      18.83
     3      18.93      63.42
     4       2.01      18.49
     5      53.64      51.79
     6       2.77      35.00
     7      12.04      56.48
     8      61.69      48.82
     9      24.04      95.43
    10      53.35       4.22
  ... (40 więcej)

Ceny przedmiot × miasto (pierwsze 10 × 6):
              c1      c2      c3      c4      c5      c6
  it 1    96.80  146.82   64.21   77.21  149.05   85.42
  it 2   134.03  106.27  142.66  100.03   93.91   65.55
  it 3    98.94  142.52  122.74  109.40   55.12   78.69
  it 4   146.01   75.23   93.46   51.70  103.90  139.35
  it 5    52.62   73.22   87.29   87.31   59.55  126.45
  it 6   146.29   58.29  141.53  142.85   76.22  143.22
  it 7    59.31  101.23   57.13   73.89   63.96   63.99
  it 8   126.21   54.56  143.37   77.70   72.65  134.65
  it 9    95.21   86.69  127.81  134.60   51.94   66.98
  it10   147.17   92.61   93.58   60.10  125.81   87.77
  ...
```



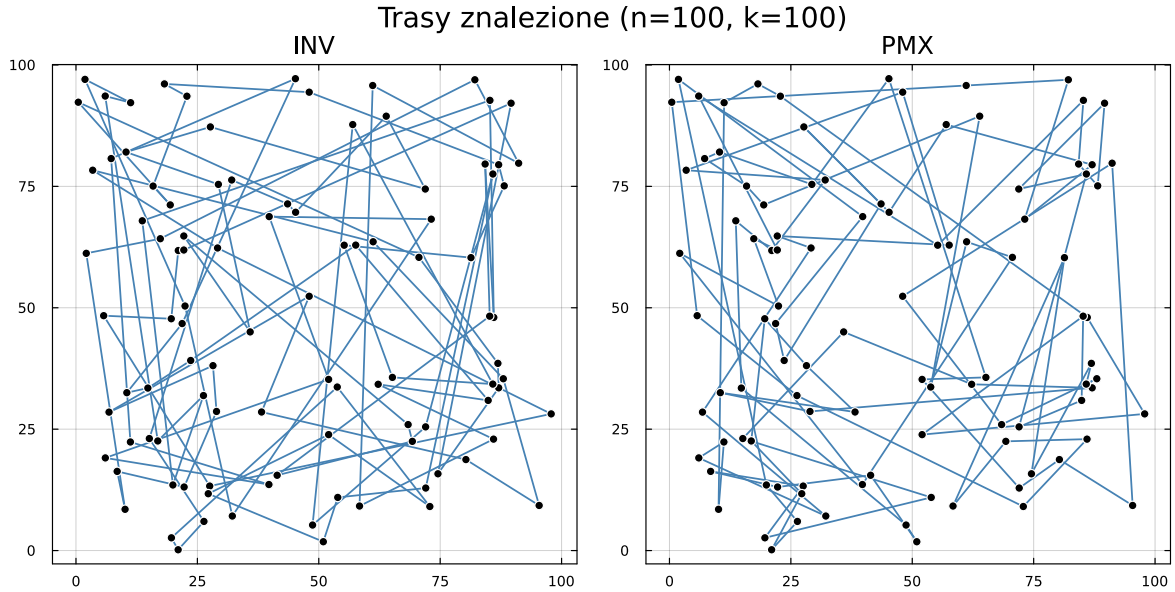

```
n=50 k=50  INV: koszt=3719.00 t=0.28s   PMX: koszt=3885.20 t=0.05s
```



```
=== Problem n=100 k=100 seed=23 ===
Liczba miast n = 100,  liczba przedmiotów k = 100
Odległości:    min=0.89   średnia=52.99   max=128.24
Ceny:          min=50.00   średnia=99.53   max=149.95

Współrzędne miast (pierwsze 10):
    id          X          Y
     1      38.22      28.53
     2       8.48      16.31
     3       7.25      80.72
     4      29.32      75.41
     5      22.20      64.78
     6      19.42      71.18
     7      97.86      28.14
     8      27.54      13.28
     9      89.60      92.12
    10      16.84      22.57
  ... (90 więcej)

Ceny przedmiot × miasto (pierwsze 10 × 6):
              c1      c2      c3      c4      c5      c6
  it 1   102.00   57.59   58.02  109.68  117.42   53.19
  it 2    54.91   93.89  112.55  137.88   78.62   99.21
  it 3   116.67  128.33   57.41   89.94  114.07  122.44
  it 4    75.22  134.81   63.48  120.11   73.73   78.08
  it 5   137.89   60.68   81.03  111.58  139.78  109.04
  it 6    52.27  111.88   55.84   82.35   66.16   67.44
  it 7    76.57  115.91  122.46  135.35  121.01  111.58
  it 8    60.95   52.13  101.79   72.76  148.11   77.21
  it 9    94.30   63.14  111.05  106.24  128.25  118.74
  it10    84.26   93.39  105.97  120.84   98.05   99.83
  ...
```




```
n=100 k=100  INV: koszt=8453.96 t=0.40s   PMX: koszt=7950.38 t=0.58s
```



```
=== Tabela (4.0 - PMX vs INV) ===
   n    k  seed |    INV koszt   INV t[s] |    PMX koszt   PMX t[s] |
    ↪  PMX/INV
  50   20    21 |      2396.34       0.21 |      2191.45       0.31 |
   ↪  0.914
  50   50    22 |      3719.00       0.28 |      3885.20       0.05 |
   ↪  1.045
 100  100    23 |      8453.96       0.40 |      7950.38       0.58 |
  ↪  0.940
```

Część na 5.0

```julia
function generate_random_tpp_selective(n, k; random_seed=1, min_cities=3,
 ↪  max_cities=n - 2)
    rng = Random.MersenneTwister(random_seed)
    X = 100 * rand(rng, n)
    Y = 100 * rand(rng, n)
    d = [sqrt((X[i] - X[j])^2 + (Y[i] - Y[j])^2) for i in 1:n, j in 1:n]
    cost_item_city = 50 .+ 100 .* rand(rng, k, n)

    for city in max(1, min_cities + 1):n
        cost_item_city[:, city] .*= (1 + 5 * rand(rng))
    end

    return X, Y, d, cost_item_city
end
```

```
generate_random_tpp_selective (generic function with 1 method)
```

```julia
function plot_ga_tour_fixed(X, Y, perm::Vector; title="",
 ↪  visited_cities=nothing)
    plt = Plots.plot(; title=title, legend=false, framestyle=:box,
 ↪  aspect_ratio=:equal)

    if visited_cities === nothing
        visited = Int[]
        visited_set = Set{Int}()
        current = 1
        for _ in 1:(length(perm)+1)
            if current in visited_set
```

```
                break
            end
            push!(visited, current)
            push!(visited_set, current)
            current = perm[current]
        end
        visited_cities = visited
    end


    for i in 1:length(visited_cities)
        j = (i % length(visited_cities)) + 1
        city_i = visited_cities[i]
        city_j = visited_cities[j]
        Plots.plot!(plt, [X[city_i], X[city_j]], [Y[city_i], Y[city_j]];
            color=:steelblue, linewidth=1.5, label="")
    end

    Plots.scatter!(plt, X, Y; color=:lightgray, markersize=4,
↪   markerstrokecolor=:white, label="")

    visited_x = X[visited_cities]
    visited_y = Y[visited_cities]
    Plots.scatter!(plt, visited_x, visited_y; color=:red, markersize=6,
                   markerstrokecolor=:darkred, label="")

    # # Zaznacz początek
    # Plots.scatter!(plt, [X[visited_cities[1]]], [Y[visited_cities[1]]],
    #                color=:green, markersize=8,
     ↪  markerstrokecolor=:darkgreen, label="")

    return plt
end
```

```
plot_ga_tour_fixed (generic function with 1 method)
```

```
function analyze_solution(perm::Vector{Int}, cost_item_city, d, item_weights,
↪   n, k; use_weights=false, weight_factor=nothing)
    visited_cities = get_visited_cities(perm, n)

    println("ANALIZA ROZWIĄZANIA")
```

```
    println("\nOdwiedzane miasta: $(visited_cities)
     ↪  ($(length(visited_cities))/$(n))")

    if use_weights && weight_factor !== nothing
        println("\nPrzedmioty kupowane w którym mieście (z KOSZTEM
         ↪  TRANSPORTU):")
    else
        println("\nPrzedmioty kupowane w którym mieście:")
    end

    if use_weights && weight_factor !== nothing
        @printf "%6s | %10s | %10s | %15s | %10s\n" "Item" "City" "Buy Cost"
         ↪  "Transport Cost" "Total"
    else
        @printf "%6s | %10s | %10s\n" "Item" "City" "Buy Cost"
    end

    total_buy_cost = 0.0
    total_transport_cost = 0.0
    purchases_by_city = Dict{Int, Vector{Int}}()

    for item in 1:k
        min_cost = Inf
        best_city = 1
        best_city_idx = 1

        for (idx, city) in enumerate(visited_cities)
            if cost_item_city[item, city] < min_cost
                min_cost = cost_item_city[item, city]
                best_city = city
                best_city_idx = idx
            end
        end

        if best_city   keys(purchases_by_city)
            purchases_by_city[best_city] = Int[]
        end
        push!(purchases_by_city[best_city], item)

        total_buy_cost += min_cost

        transport_cost = 0.0
```

```
        if use_weights && weight_factor !== nothing
            transport_distance = 0.0
            current_idx = best_city_idx
            for step in 1:length(visited_cities)
                next_idx = (current_idx % length(visited_cities)) + 1
                current_city = visited_cities[current_idx]
                next_city = visited_cities[next_idx]
                transport_distance += d[current_city, next_city]
                current_idx = next_idx
            end
            transport_cost = transport_distance * item_weights[item] *
↪   weight_factor
            total_transport_cost += transport_cost

            @printf "%6d | %10d | %10.2f | %15.2f | %10.2f\n" item best_city
             ↪  min_cost transport_cost (min_cost + transport_cost)
        else
            @printf "%6d | %10d | %10.2f\n" item best_city min_cost
        end
    end

    println("\nPodsumowanie zakupów po miastach:")
    for city in sort(collect(keys(purchases_by_city)))
        items = purchases_by_city[city]
        @printf "Miasto %3d: kupuje %3d przedmiotów (items: " city
         ↪  length(items)
        print(join(items[1:min(5, length(items))], ", "))
        if length(items) > 5
            print(", ... ($(length(items)-5) więcej)")
        end
        println(")")
    end

    tour_cost = 0.0
    for i in 1:length(visited_cities)
        j = (i % length(visited_cities)) + 1
        city_i = visited_cities[i]
        city_j = visited_cities[j]
        tour_cost += d[city_i, city_j]
    end

    println("KOSZT CAŁKOWITY")
```

```
    @printf "Koszt trasy:        %12.2f\n" tour_cost
    @printf "Koszt przedmiotów:  %12.2f\n" total_buy_cost
    if use_weights && weight_factor !== nothing
        @printf "Koszt transportu:   %12.2f\n" total_transport_cost
        @printf "RAZEM:              %12.2f\n" (tour_cost + total_buy_cost +
         ↪  total_transport_cost)
    else
        @printf "RAZEM:              %12.2f\n" (tour_cost + total_buy_cost)
    end

    return visited_cities, purchases_by_city, tour_cost, total_buy_cost,
     ↪  total_transport_cost
end
```

```
analyze_solution (generic function with 1 method)
```

```
function get_visited_cities(perm::Vector{Int}, n::Int)
    visited = Int[]
    visited_set = Set{Int}()
    current = 1

    for _ in 1:(n+1)
        if current in visited_set
            break
        end
        push!(visited, current)
        push!(visited_set, current)
        if current > length(perm) || current < 1
            break
        end
        current = perm[current]
    end

    if length(visited) < 2
        visited = [1, 2]
    end

    return visited
end
```

```
get_visited_cities (generic function with 1 method)
```

Modyfikacja algorytmu genetycznego dodająca możliwość omijania miast

Funkcje obliczania kosztu

- wersja bez wag: koszt = długość trasy + koszty przedmiotów (wybrane najtańsze miasta z pośród odwiedzonych)
- wersja z wagami: koszt = trasa + koszty przedmiotów + długość trasy * avg_weght * weight_factor

```
function make_cost_tpp_selective(d, cost_item_city)
    n = size(d, 1)
    k = size(cost_item_city, 1)

    function cost(perm::Vector{Int})
        visited_cities = get_visited_cities(perm, n)

        if length(visited_cities) < 2
            return Inf
        end

        tour_cost = 0.0
        for i in 1:length(visited_cities)
            j = (i % length(visited_cities)) + 1
            city_i = visited_cities[i]
            city_j = visited_cities[j]
            tour_cost += d[city_i, city_j]
        end

        item_cost = 0.0
        for item in 1:k
            min_cost = Inf
            for city in visited_cities
                min_cost = min(min_cost, cost_item_city[item, city])
            end
            item_cost += min_cost
        end

        return tour_cost + item_cost
    end

    return cost
end
```

```julia
function calculate_weight_factor(d, cost_item_city, item_weights,
↪   avg_price_percentage=0.01)

    avg_weight = mean(item_weights)
    max_distance = maximum(d)
    avg_item_price = mean(cost_item_city)
    weight_factor = (avg_price_percentage * avg_item_price) / (avg_weight *
↪   max_distance)

    return weight_factor
end

function make_cost_tpp_selective_with_weight(d, cost_item_city, item_weights;
↪   weight_percentage=0.01)

    n = size(d, 1)
    k = size(cost_item_city, 1)

    weight_factor = calculate_weight_factor(d, cost_item_city, item_weights,
↪   weight_percentage)

    function cost(perm::Vector{Int})
        visited_cities = get_visited_cities(perm, n)

        if length(visited_cities) < 2
            return Inf
        end

        tour_cost = 0.0
        for i in 1:length(visited_cities)
            j = (i % length(visited_cities)) + 1
            city_i = visited_cities[i]
            city_j = visited_cities[j]
            tour_cost += d[city_i, city_j]
        end

        item_cost = 0.0
        for item in 1:k
            min_cost = Inf
            best_city = 1
            best_city_idx = 1
```

```
            for (idx, city) in enumerate(visited_cities)
                if cost_item_city[item, city] < min_cost
                    min_cost = cost_item_city[item, city]
                    best_city = city
                    best_city_idx = idx
                end
            end

            transport_distance = 0.0

            current_idx = best_city_idx
            for step in 1:length(visited_cities)
                next_idx = (current_idx % length(visited_cities)) + 1
                current_city = visited_cities[current_idx]
                next_city = visited_cities[next_idx]

                transport_distance += d[current_city, next_city]

                current_idx = next_idx
            end

            transport_cost = transport_distance * item_weights[item] *
↪   weight_factor

            item_cost += min_cost + transport_cost
        end

        return tour_cost + item_cost
    end

    return cost
end
```

```
make_cost_tpp_selective_with_weight (generic function with 1 method)
```

Zmienione funkcje do algorytmu genetycznego tak aby pomijał miasta.

```
function recombine_selective_order(parent1::AbstractVector{Int},
↪   parent2::AbstractVector{Int};
                                   p::Float64=0.5,
↪   rng::AbstractRNG=Random.default_rng())
    n = length(parent1)
```

```
    point1 = rand(rng, 1:n-1)
    point2 = rand(rng, point1+1:n)

    child = copy(parent1)
    segment = parent2[point1:point2]
    child[point1:point2] = segment

    seen = Set(segment)
    for i in vcat(1:point1-1, point2+1:n)
        if child[i] in seen
            # Znajdź brakujące miasto
            for city in 1:n
                if city   seen
                    child[i] = city
                    push!(seen, city)
                    break
                end
            end
        else
            push!(seen, child[i])
        end
    end

    return child
end

function recombine_selective_tour(parent1::AbstractVector{Int},
 ↪  parent2::AbstractVector{Int};
    p::Float64=0.5, rng::AbstractRNG=Random.default_rng())
    child1 = recombine_selective_order(parent1, parent2; p=p, rng=rng)
    child2 = recombine_selective_order(parent2, parent1; p=p, rng=rng)
    return [child1, child2]
end

function mutate_selective_swap(individual::AbstractVector{Int};
 ↪  p::Float64=0.1,
                                rng::AbstractRNG=Random.default_rng())
    if rand(rng) < p
        i = rand(rng, 1:length(individual))
        j = rand(rng, 1:length(individual))
        individual[i], individual[j] = individual[j], individual[i]
    end
```

```
    return individual
end

function mutate_selective_reverse(individual::AbstractVector{Int};
 ↪  p::Float64=0.05,
                                  rng::AbstractRNG=Random.default_rng())
    if rand(rng) < p
        i = rand(rng, 1:length(individual)-1)
        j = rand(rng, i+1:length(individual))
        reverse!(individual[i:j])
    end
    return individual
end

function mutate_selective_reorder(individual::AbstractVector{Int};
 ↪  p::Float64=0.15,
                                  rng::AbstractRNG=Random.default_rng())
    if rand(rng) < p
        i = rand(rng, 1:length(individual))
        j = rand(rng, 1:length(individual))
        if i != j
            city = splice!(individual, i)
            insert!(individual, j, city)
        end
    end
    return individual
end

function mutate_selective(individual::AbstractVector{Int}; p::Float64=0.2,
                          rng::AbstractRNG=Random.default_rng())
    mutate_selective_swap(individual; p=0.1, rng=rng)
    mutate_selective_reverse(individual; p=0.05, rng=rng)
    mutate_selective_reorder(individual; p=0.08, rng=rng)
    return individual
end
```

```
mutate_selective (generic function with 1 method)
```

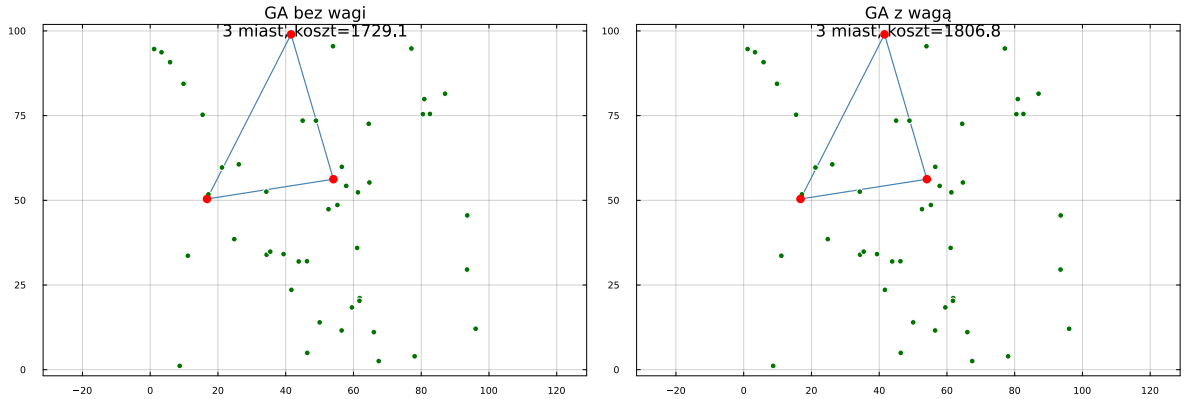
```
results_ga_selective = Any[]

for (test_idx, (n, k, seed)) in enumerate([(50, 20, 51), (50, 50, 52), (100,
 ↪  100, 53)])
```

```
    println("Test $(test_idx): n=$n, k=$k, seed=$seed - GA Selective")

    X, Y, d, cost_item_city = generate_random_tpp_selective(n, k;
↪   random_seed=seed)


    rng = Random.MersenneTwister(seed + 100)
    item_weights = 0.5 .+ 2.0 .* rand(rng, k)

    println("\n[GA - bez uwzględnienia wagi]")
    cost_fn = make_cost_tpp_selective(d, cost_item_city)

    x0 = [mod1(i + 1, n) for i in 1:n]
    pop = (n <= 50 ? 500 : 300)
    population = [randperm(n) for _ in 1:pop]

    opts = Evolutionary.Options(; iterations=3000, time_limit=55.0)
    ga_selective = GA(;
        populationSize=pop,
        crossover=recombine_selective_tour,
        mutation=mutate_selective,
        epsilon=0.05,
        crossoverRate=0.9
    )

    t_ga_sel = @elapsed res_ga_sel = Evolutionary.optimize(cost_fn, x0,
↪   ga_selective, opts)
    println(" Koszt: $(res_ga_sel.minimum)  Czas: $t_ga_sel s   Iteracje:
     ↪  $(res_ga_sel.iterations)")

    visited_no_weight, purchases_no_weight, tour_cost_no_weight,
↪   item_cost_no_weight, _ =
        analyze_solution(res_ga_sel.minimizer, cost_item_city, d,
         ↪  item_weights, n, k; use_weights=false)


    println("\n[GA - z uwzględnieniem wagi przedmiotów]")
    cost_fn_weight = make_cost_tpp_selective_with_weight(d, cost_item_city,
↪   item_weights;

↪   weight_percentage=0.01)
```

```
    t_ga_sel_w = @elapsed res_ga_sel_w =
↪   Evolutionary.optimize(cost_fn_weight, x0, ga_selective, opts)
    println(" Koszt: $(res_ga_sel_w.minimum)  Czas: $t_ga_sel_w s")

    weight_factor = calculate_weight_factor(d, cost_item_city, item_weights,
↪   0.01)
    visited_with_weight, purchases_with_weight, tour_cost_with_weight,
↪   item_cost_with_weight, transport_cost_with_weight =
        analyze_solution(res_ga_sel_w.minimizer, cost_item_city, d,
         ↪  item_weights, n, k;
                        use_weights=true, weight_factor=weight_factor)


    println("PORÓWNANIE WERSJI")
    @printf "Bez wagi:   odwiedzono %3d miast (z %d dostępnych)\n"
     ↪  length(visited_no_weight) n
    @printf "Z wagą:     odwiedzono %3d miast (z %d dostępnych)\n"
     ↪  length(visited_with_weight) n

    display(Plots.plot(
        plot_ga_tour_fixed(X, Y, res_ga_sel.minimizer;
                           title="GA bez wagi\n$(length(visited_no_weight))
↪   miast, koszt=$(round(res_ga_sel.minimum, digits=1))",
                           visited_cities=visited_no_weight),
        plot_ga_tour_fixed(X, Y, res_ga_sel_w.minimizer;
                           title="GA z wagą\n$(length(visited_with_weight))
↪   miast, koszt=$(round(res_ga_sel_w.minimum, digits=1))",
                           visited_cities=visited_with_weight),
        layout=(1, 2), size=(1500, 500)
    ))

    push!(results_ga_selective, (
        n=n, k=k, seed=seed,
        ga_cost_no_weight=res_ga_sel.minimum,
        ga_time_no_weight=t_ga_sel,
        ga_cities_no_weight=length(visited_no_weight),
        ga_cost_with_weight=res_ga_sel_w.minimum,
        ga_time_with_weight=t_ga_sel_w,
        ga_cities_with_weight=length(visited_with_weight)
    ))
end
```

```
Test 1: n=50, k=20, seed=51 - GA Selective

[GA - bez uwzględnienia wagi]
 Koszt: 1729.061948657135  Czas: 0.017975178 s   Iteracje: 13
ANALIZA ROZWIĄZANIA

Odwiedzane miasta: [1, 2, 3] (3/50)

Przedmioty kupowane w którym mieście:
  Item |       City |   Buy Cost
     1 |          1 |      80.00
     2 |          3 |      84.79
     3 |          1 |      53.70
     4 |          1 |      60.91
     5 |          1 |      53.82
     6 |          3 |     110.24
     7 |          1 |      89.24
     8 |          3 |      64.62
     9 |          3 |      94.95
    10 |          3 |      72.24
    11 |          1 |      76.99
    12 |          1 |      85.93
    13 |          1 |      99.19
    14 |          2 |      80.51
    15 |          3 |     111.70
    16 |          3 |      58.41
    17 |          1 |      95.00
    18 |          2 |      66.76
    19 |          2 |      50.90
    20 |          2 |     102.29

Podsumowanie zakupów po miastach:
Miasto   1: kupuje   9 przedmiotów (items: 1, 3, 4, 5, 7, ... (4 więcej))
Miasto   2: kupuje   4 przedmiotów (items: 14, 18, 19, 20)
Miasto   3: kupuje   7 przedmiotów (items: 2, 6, 8, 9, 10, ... (2 więcej))
KOSZT CAŁKOWITY
Koszt trasy:              136.86
Koszt przedmiotów:       1592.20
RAZEM:                   1729.06

[GA - z uwzględnieniem wagi przedmiotów]
 Koszt: 1806.7947580244122  Czas: 0.028202013 s
ANALIZA ROZWIĄZANIA
```

```
Odwiedzane miasta: [1, 2, 3] (3/50)

Przedmioty kupowane w którym mieście (z KOSZTEM TRANSPORTU):
  Item |       City |   Buy Cost |  Transport Cost |      Total
     1 |          1 |      80.00 |            1.53 |      81.54
     2 |          3 |      84.79 |            1.41 |      86.19
     3 |          1 |      53.70 |            4.41 |      58.12
     4 |          1 |      60.91 |            2.41 |      63.32
     5 |          1 |      53.82 |            3.35 |      57.18
     6 |          3 |     110.24 |            6.67 |     116.91
     7 |          1 |      89.24 |            4.10 |      93.34
     8 |          3 |      64.62 |            4.38 |      69.00
     9 |          3 |      94.95 |            1.38 |      96.33
    10 |          3 |      72.24 |            5.77 |      78.02
    11 |          1 |      76.99 |            5.94 |      82.93
    12 |          1 |      85.93 |            3.94 |      89.87
    13 |          1 |      99.19 |            3.20 |     102.39
    14 |          2 |      80.51 |            5.82 |      86.33
    15 |          3 |     111.70 |            2.01 |     113.71
    16 |          3 |      58.41 |            5.22 |      63.63
    17 |          1 |      95.00 |            3.59 |      98.59
    18 |          2 |      66.76 |            3.09 |      69.84
    19 |          2 |      50.90 |            6.47 |      57.37
    20 |          2 |     102.29 |            3.04 |     105.34

Podsumowanie zakupów po miastach:
Miasto   1: kupuje   9 przedmiotów (items: 1, 3, 4, 5, 7, ... (4 więcej))
Miasto   2: kupuje   4 przedmiotów (items: 14, 18, 19, 20)
Miasto   3: kupuje   7 przedmiotów (items: 2, 6, 8, 9, 10, ... (2 więcej))
KOSZT CAŁKOWITY
Koszt trasy:              136.86
Koszt przedmiotów:       1592.20
Koszt transportu:          77.73
RAZEM:                   1806.79
PORÓWNANIE WERSJI
Bez wagi:   odwiedzono   3 miast (z 50 dostępnych)
Z wagą:     odwiedzono   3 miast (z 50 dostępnych)
```



Test 2: n=50, k=50, seed=52 - GA Selective

[GA - bez uwzględnienia wagi]
 Koszt: 3772.366398830682 Czas: 0.053223747 s Iteracje: 17
ANALIZA ROZWIĄZANIA

Odwiedzane miasta: [1, 2, 3, 4] (4/50)

Przedmioty kupowane w którym mieście:

Item	City	Buy Cost
1	1	51.58
2	3	81.66
3	3	67.32
4	1	74.20
5	2	61.22
6	3	81.41
7	2	69.48
8	2	52.97
9	1	68.56
10	1	62.46
11	2	50.92
12	2	50.80
13	4	75.83
14	2	57.05
15	2	55.55
16	3	68.01
17	2	83.46
18	1	54.09
19	2	119.33
20	2	83.80
21	4	101.67

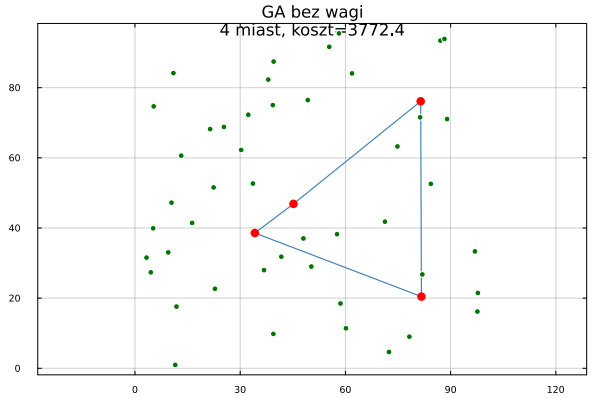
```
    22 |          3 |      56.14
    23 |          1 |      66.44
    24 |          2 |      63.93
    25 |          3 |      61.89
    26 |          1 |      71.44
    27 |          3 |     115.75
    28 |          2 |     114.53
    29 |          2 |      66.45
    30 |          2 |      86.44
    31 |          3 |      50.86
    32 |          4 |      77.76
    33 |          1 |      73.34
    34 |          2 |      67.93
    35 |          1 |      72.25
    36 |          3 |      80.35
    37 |          3 |      98.59
    38 |          1 |      58.31
    39 |          4 |      75.27
    40 |          3 |     114.01
    41 |          3 |      51.02
    42 |          1 |      90.75
    43 |          3 |      51.74
    44 |          3 |      53.16
    45 |          4 |      73.61
    46 |          1 |      57.10
    47 |          2 |      56.29
    48 |          1 |      70.17
    49 |          2 |      90.93
    50 |          3 |      67.64

Podsumowanie zakupów po miastach:
Miasto   1: kupuje  13 przedmiotów (items: 1, 4, 9, 10, 18, ... (8 więcej))
Miasto   2: kupuje  17 przedmiotów (items: 5, 7, 8, 11, 12, ... (12 więcej))
Miasto   3: kupuje  15 przedmiotów (items: 2, 3, 6, 16, 22, ... (10 więcej))
Miasto   4: kupuje   5 przedmiotów (items: 13, 21, 32, 39, 45)
KOSZT CAŁKOWITY
Koszt trasy:              166.89
Koszt przedmiotów:       3605.47
RAZEM:                   3772.37

[GA - z uwzględnieniem wagi przedmiotów]
 Koszt: 3995.2877962912135  Czas: 0.054783968 s
ANALIZA ROZWIĄZANIA
```

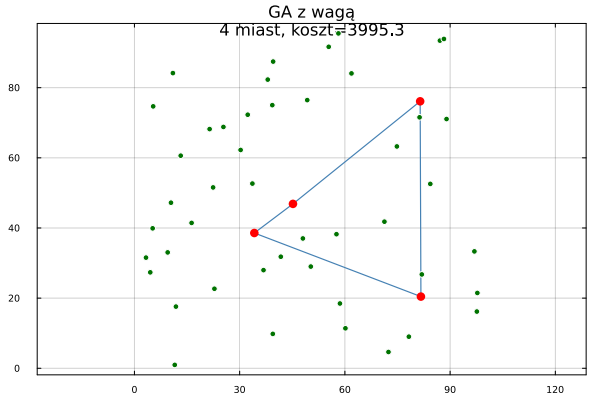
Odwiedzane miasta: [1, 2, 3, 4] (4/50)

Przedmioty kupowane w którym mieście (z KOSZTEM TRANSPORTU):

Item	City	Buy Cost	Transport Cost	Total
1	1	51.58	2.24	53.82
2	3	81.66	6.93	88.59
3	3	67.32	6.11	73.43
4	1	74.20	3.35	77.55
5	2	61.22	6.59	67.81
6	3	81.41	4.94	86.36
7	2	69.48	4.45	73.93
8	2	52.97	2.25	55.22
9	1	68.56	6.51	75.07
10	1	62.46	4.23	66.69
11	2	50.92	6.79	57.71
12	2	50.80	2.72	53.52
13	4	75.83	3.67	79.50
14	2	57.05	5.53	62.58
15	2	55.55	5.08	60.64
16	3	68.01	6.60	74.61
17	2	83.46	2.57	86.03
18	1	54.09	5.98	60.07
19	2	119.33	3.85	123.18
20	2	83.80	5.33	89.13
21	4	101.67	7.17	108.84
22	3	56.14	2.76	58.89
23	1	66.44	4.59	71.03
24	2	63.93	5.56	69.48
25	3	61.89	3.75	65.64
26	1	71.44	3.91	75.35
27	3	115.75	6.89	122.64
28	2	114.53	7.30	121.83
29	2	66.45	6.54	72.99
30	2	86.44	2.96	89.41
31	3	50.86	6.19	57.05
32	4	77.76	5.02	82.78
33	1	73.34	6.32	79.67
34	2	67.93	1.82	69.75
35	1	72.25	5.25	77.50
36	3	80.35	2.39	82.73
37	3	98.59	1.99	100.58
38	1	58.31	2.89	61.20

```
    39 |          4 |      75.27 |            2.09 |      77.36
    40 |          3 |     114.01 |            2.19 |     116.20
    41 |          3 |      51.02 |            3.88 |      54.90
    42 |          1 |      90.75 |            4.40 |      95.16
    43 |          3 |      51.74 |            1.79 |      53.53
    44 |          3 |      53.16 |            3.46 |      56.62
    45 |          4 |      73.61 |            5.60 |      79.20
    46 |          1 |      57.10 |            6.18 |      63.28
    47 |          2 |      56.29 |            5.88 |      62.17
    48 |          1 |      70.17 |            3.19 |      73.36
    49 |          2 |      90.93 |            3.52 |      94.45
    50 |          3 |      67.64 |            1.72 |      69.36

Podsumowanie zakupów po miastach:
Miasto   1: kupuje  13 przedmiotów (items: 1, 4, 9, 10, 18, ... (8 więcej))
Miasto   2: kupuje  17 przedmiotów (items: 5, 7, 8, 11, 12, ... (12 więcej))
Miasto   3: kupuje  15 przedmiotów (items: 2, 3, 6, 16, 22, ... (10 więcej))
Miasto   4: kupuje   5 przedmiotów (items: 13, 21, 32, 39, 45)
KOSZT CAŁKOWITY
Koszt trasy:              166.89
Koszt przedmiotów:       3605.47
Koszt transportu:         222.92
RAZEM:                   3995.29
PORÓWNANIE WERSJI
Bez wagi:   odwiedzono   4 miast (z 50 dostępnych)
Z wagą:     odwiedzono   4 miast (z 50 dostępnych)
```






```
Test 3: n=100, k=100, seed=53 - GA Selective

[GA - bez uwzględnienia wagi]
 Koszt: 7504.801590786862  Czas: 0.078304383 s   Iteracje: 20
```

ANALIZA ROZWIĄZANIA

Odwiedzane miasta: [1, 2, 3, 5, 7, 8, 9, 10, 11, 12, 13] (11/100)

Przedmioty kupowane w którym mieście:

Item	City	Buy Cost
1	2	76.76
2	3	63.74
3	12	82.59
4	1	53.44
5	3	61.80
6	12	75.61
7	12	80.79
8	12	93.58
9	1	68.45
10	1	58.85
11	3	58.74
12	2	78.25
13	3	57.29
14	3	63.46
15	1	53.13
16	3	52.45
17	3	53.28
18	1	69.22
19	1	58.87
20	2	50.12
21	3	63.98
22	13	80.71
23	2	58.88
24	1	56.07
25	12	62.78
26	12	85.32
27	3	68.88
28	13	89.41
29	1	88.95
30	2	59.89
31	12	72.43
32	3	69.07
33	2	72.36
34	1	53.92
35	1	54.67
36	3	54.27
37	1	61.58

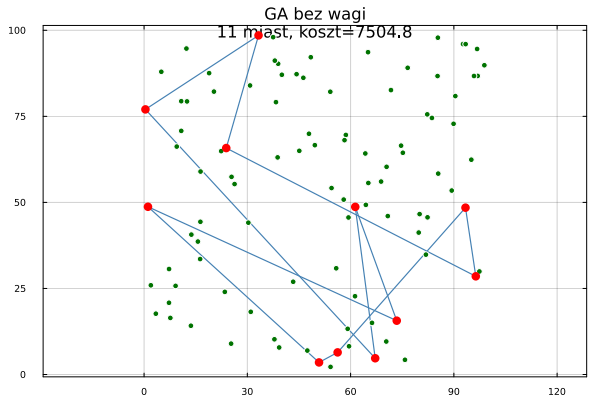
38	1	63.48
39	2	54.89
40	3	83.78
41	3	61.94
42	13	86.80
43	12	56.63
44	3	67.71
45	1	56.14
46	3	99.35
47	1	72.78
48	12	90.99
49	1	50.94
50	3	52.20
51	1	63.80
52	2	50.38
53	12	61.08
54	12	58.41
55	1	57.31
56	8	101.58
57	12	95.46
58	3	53.15
59	2	67.41
60	3	61.88
61	2	93.81
62	3	101.55
63	3	95.80
64	3	72.24
65	12	59.52
66	1	55.85
67	3	55.86
68	1	77.12
69	12	63.61
70	12	74.36
71	12	55.86
72	2	82.98
73	1	61.93
74	1	50.72
75	3	95.33
76	1	72.08
77	3	50.38
78	3	55.83
79	1	50.33
80	1	116.73

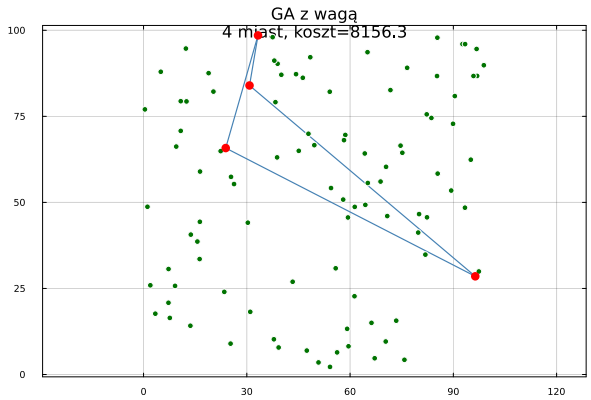
```
    81 |          3 |      57.38
    82 |          3 |      90.59
    83 |          1 |      63.29
    84 |          2 |      52.08
    85 |          3 |      50.69
    86 |         13 |     100.48
    87 |          1 |      76.71
    88 |         13 |     100.12
    89 |          1 |      70.78
    90 |          2 |      61.85
    91 |         12 |      55.32
    92 |          1 |      84.29
    93 |          2 |      90.88
    94 |          3 |      89.44
    95 |          2 |      56.62
    96 |          1 |      53.82
    97 |          3 |      72.72
    98 |         12 |      55.89
    99 |         12 |     120.20
   100 |         12 |      66.20

Podsumowanie zakupów po miastach:
Miasto   1: kupuje  29 przedmiotów (items: 4, 9, 10, 15, 18, ... (24 więcej))
Miasto   2: kupuje  15 przedmiotów (items: 1, 12, 20, 23, 30, ... (10
 ↪  więcej))
Miasto   3: kupuje  30 przedmiotów (items: 2, 5, 11, 13, 14, ... (25 więcej))
Miasto   8: kupuje   1 przedmiotów (items: 56)
Miasto  12: kupuje  20 przedmiotów (items: 3, 6, 7, 8, 25, ... (15 więcej))
Miasto  13: kupuje   5 przedmiotów (items: 22, 28, 42, 86, 88)
KOSZT CAŁKOWITY
Koszt trasy:              561.86
Koszt przedmiotów:       6942.94
RAZEM:                   7504.80

[GA - z uwzględnieniem wagi przedmiotów]
 Koszt: 8156.257233216303  Czas: 0.193717581 s
ANALIZA ROZWIĄZANIA

Odwiedzane miasta: [1, 2, 3, 4] (4/100)

Przedmioty kupowane w którym mieście (z KOSZTEM TRANSPORTU):
  Item |       City |   Buy Cost |  Transport Cost |      Total
     1 |          2 |      76.76 |            4.58 |      81.34
```

2	3	63.74	5.30	69.03
3	3	107.80	3.01	110.81
4	1	53.44	8.15	61.58
5	3	61.80	7.34	69.14
6	3	85.77	7.98	93.75
7	3	119.88	6.31	126.19
8	2	106.21	5.19	111.41
9	1	68.45	9.47	77.92
10	1	58.85	2.92	61.77
11	3	58.74	7.82	66.56
12	2	78.25	6.40	84.65
13	3	57.29	6.84	64.12
14	3	63.46	2.10	65.56
15	1	53.13	7.44	60.57
16	3	52.45	9.72	62.17
17	3	53.28	5.67	58.95
18	1	69.22	7.21	76.44
19	1	58.87	8.88	67.76
20	2	50.12	9.04	59.16
21	3	63.98	9.69	73.68
22	2	85.83	9.15	94.98
23	2	58.88	4.15	63.02
24	1	56.07	6.48	62.55
25	1	65.10	9.42	74.51
26	3	88.38	8.19	96.57
27	3	68.88	3.99	72.87
28	2	91.23	8.10	99.33
29	1	88.95	5.87	94.83
30	2	59.89	2.62	62.51
31	1	83.17	4.89	88.06
32	3	69.07	3.12	72.19
33	2	72.36	8.63	80.99
34	1	53.92	3.08	57.00
35	1	54.67	7.38	62.05
36	3	54.27	5.95	60.22
37	1	61.58	7.68	69.26
38	1	63.48	6.93	70.41
39	2	54.89	6.13	61.01
40	3	83.78	4.65	88.43
41	3	61.94	3.99	65.93
42	3	97.59	9.87	107.46
43	3	85.45	2.86	88.31
44	3	67.71	8.16	75.87

45	1	56.14	9.78	65.93
46	3	99.35	4.34	103.69
47	1	72.78	2.63	75.40
48	1	104.50	5.32	109.82
49	1	50.94	5.05	55.98
50	3	52.20	4.95	57.15
51	1	63.80	4.78	68.58
52	2	50.38	5.77	56.15
53	3	84.72	3.34	88.06
54	2	65.58	7.81	73.39
55	1	57.31	2.07	59.37
56	1	121.10	5.18	126.28
57	2	99.32	7.21	106.53
58	3	53.15	2.40	55.55
59	2	67.41	4.69	72.10
60	3	61.88	5.04	66.92
61	2	93.81	3.47	97.28
62	3	101.55	7.56	109.11
63	3	95.80	8.70	104.51
64	3	72.24	6.69	78.93
65	2	109.02	2.12	111.15
66	1	55.85	3.51	59.36
67	3	55.86	4.96	60.82
68	1	77.12	8.38	85.50
69	1	67.49	9.87	77.36
70	3	80.98	8.24	89.23
71	1	56.27	4.43	60.70
72	2	82.98	5.48	88.46
73	1	61.93	2.15	64.08
74	1	50.72	6.54	57.26
75	3	95.33	4.50	99.83
76	1	72.08	5.02	77.10
77	3	50.38	2.86	53.24
78	3	55.83	7.60	63.43
79	1	50.33	3.02	53.35
80	1	116.73	5.51	122.24
81	3	57.38	9.39	66.78
82	3	90.59	7.44	98.03
83	1	63.29	5.13	68.42
84	2	52.08	9.88	61.96
85	3	50.69	9.33	60.02
86	2	107.55	7.74	115.30
87	1	76.71	9.32	86.03

```
  88 |          1 |     109.46 |            6.87 |     116.33
  89 |          1 |      70.78 |            4.69 |      75.47
  90 |          2 |      61.85 |            4.70 |      66.54
  91 |          3 |     122.21 |            9.58 |     131.79
  92 |          1 |      84.29 |            3.68 |      87.96
  93 |          2 |      90.88 |            5.59 |      96.47
  94 |          3 |      89.44 |            6.48 |      95.92
  95 |          2 |      56.62 |            9.13 |      65.75
  96 |          1 |      53.82 |            6.67 |      60.49
  97 |          3 |      72.72 |            4.54 |      77.26
  98 |          2 |      59.02 |            9.32 |      68.34
  99 |          3 |     124.16 |            3.35 |     127.51
 100 |          3 |      86.91 |            2.02 |      88.93

Podsumowanie zakupów po miastach:
Miasto   1: kupuje  36 przedmiotów (items: 4, 9, 10, 15, 18, ... (31 więcej))
Miasto   2: kupuje  23 przedmiotów (items: 1, 8, 12, 20, 22, ... (18 więcej))
Miasto   3: kupuje  41 przedmiotów (items: 2, 3, 5, 6, 7, ... (36 więcej))
KOSZT CAŁKOWITY
Koszt trasy:              216.20
Koszt przedmiotów:       7331.90
Koszt transportu:         608.16
RAZEM:                   8156.26
PORÓWNANIE WERSJI
Bez wagi:   odwiedzono  11 miast (z 100 dostępnych)
Z wagą:     odwiedzono   4 miast (z 100 dostępnych)
```





Lazy constraint

```
function plot_ga_tour(X, Y, perm::Vector; title="")
    plt = Plots.plot(; title=title, legend=false, framestyle=:box,
↪   aspect_ratio=:equal)

    # perm[i] = następne miasto po mieście i (reprezentacja "next-city")
    # Rysuj krawędzie w kolejności obejścia
    current = 1
    for step in 1:length(perm)
        next = perm[current]
        Plots.plot!(plt, [X[current], X[next]], [Y[current], Y[next]];
                    color=:steelblue, linewidth=1.5)
        current = next
        if current == 1  # Powróciłem do startu - koniec pętli
            break
        end
    end
    Plots.scatter!(plt, X, Y; color=:black, markersize=4,
↪   markerstrokecolor=:white)
    return plt
end
```

```
plot_ga_tour (generic function with 1 method)
```

```
function find_subtour_tsp(x_vals, n)
    # Znajdź jeden subtour zaczynając od pierwszego węzła w cyklu
    visited = falses(n)

    for start in 1:n
        visited[start] && continue

        # sprawdź czy start ma jakieś krawędzie
        any(x_vals[start, j] > 0.5 for j in 1:n) || continue

        tour = Int[]
        current = start

        for _ in 1:n
            visited[current] && break
            push!(tour, current)
```

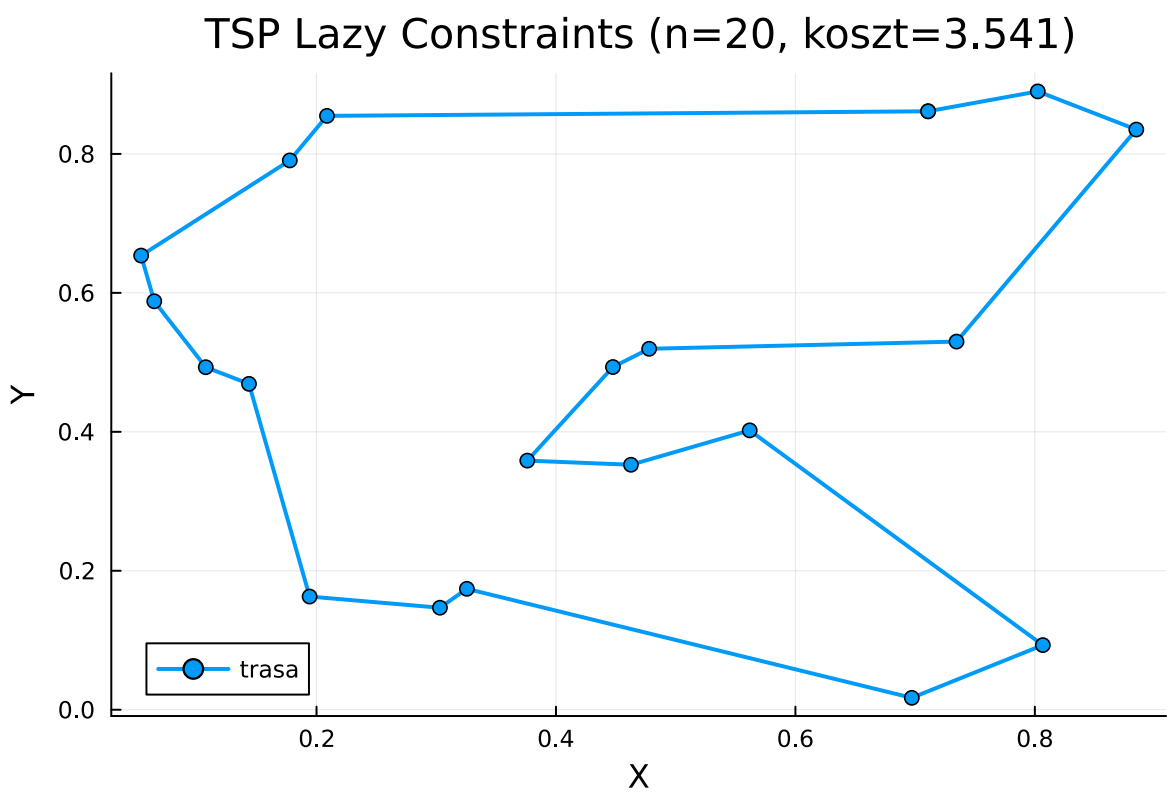
```
            visited[current] = true

            moved = false
            for j in 1:n
                if x_vals[current, j] > 0.5 && !visited[j]
                    current = j
                    moved = true
                    break
                end
            end
            moved || break
        end

        # zwróć pierwszy znaleziony subtour krótszy niż n
        if 1 < length(tour) < n
            return tour
        end
    end

    return Int[]  # brak subtour → rozwiązanie poprawne
end

function build_tsp_lazy_model(d, n)
    model = Model(GLPK.Optimizer)
    set_attribute(model, "msg_lev", GLPK.GLP_MSG_OFF)

    # x[i,j] = 1 jeśli krawędź (i,j) w trasie; Symmetric → x[i,j]==x[j,i]
    @variable(model, x[i=1:n, j=1:n], Bin, Symmetric)

    @objective(model, Min,
        sum(d[i, j] * x[i, j] for i in 1:n, j in 1:n if i < j)
    )

    # Każde miasto ma dokładnie 2 sąsiadów (wejście + wyjście)
    for i in 1:n
        @constraint(model, sum(x[i, j] for j in 1:n) == 2)
        @constraint(model, x[i, i] == 0)
    end

    function subtour_elimination_callback(cb_data)
        status = callback_node_status(cb_data, model)
        status != MOI.CALLBACK_NODE_STATUS_INTEGER && return
```

```
        x_vals = callback_value.(cb_data, x)

        cycle = find_subtour_tsp(x_vals, n)
        isempty(cycle) && return  # brak subtour → OK

        println("  Znaleziono subtour długości $(length(cycle)): $cycle")

        S = [(i, j) for i in cycle, j in cycle if i < j]
        con = @build_constraint(
            sum(x[i, j] for (i, j) in S) <= length(cycle) - 1
        )
        MOI.submit(model, MOI.LazyConstraint(cb_data), con)
    end

    set_attribute(model, MOI.LazyConstraintCallback(),
     ↪  subtour_elimination_callback)

    return model
end

function solve_tsp_lazy(n; random_seed=1, time_limit=120.0)
    # Generuj losowe miasta
    rng = Random.MersenneTwister(random_seed)
    X = rand(rng, n)
    Y = rand(rng, n)
    d = [sqrt((X[i]-X[j])^2 + (Y[i]-Y[j])^2) for i in 1:n, j in 1:n]

    model = build_tsp_lazy_model(d, n)
    set_time_limit_sec(model, time_limit)

    println("Rozwiązuję TSP (n=$n, seed=$random_seed)...")
    t_start = time()
    optimize!(model)
    t_elapsed = time() - t_start

    status = termination_status(model)
    println("Status: $status, czas: $(round(t_elapsed, digits=2)) s")

    if !is_solved_and_feasible(model)
        return (status=status, cost=Inf, tour=Int[], X=X, Y=Y, d=d,
         ↪  time=t_elapsed)
```

```
    end

    x_sol = JuMP.value.(model[:x])
    cost  = objective_value(model)

    # Rekonstrukcja trasy
    tour    = Int[]
    in_tour = Set{Int}()
    current = 1
    push!(tour, current); push!(in_tour, current)
    while true
        nxt = nothing
        for j in 1:n
            if j  in_tour && x_sol[current, j] > 0.5
                nxt = j; break
            end
        end
        nxt === nothing && break
        push!(tour, nxt); push!(in_tour, nxt)
        current = nxt
        length(tour) >= n && break
    end

    println("  Koszt: $(round(cost, digits=4))")
    println("  Trasa: $tour")

    return (status=status, cost=cost, tour=tour, X=X, Y=Y, d=d,
     ↪  time=t_elapsed)
end

using Random

res = solve_tsp_lazy(20; random_seed=42, time_limit=60.0)

if res.cost < Inf
    display(Plots.plot(
        [res.X[c] for c in [res.tour; res.tour[1]]],
        [res.Y[c] for c in [res.tour; res.tour[1]]],
        marker=:circle, linewidth=2,
        title="TSP Lazy Constraints (n=20,
 ↪  koszt=$(round(res.cost,digits=3)))",
        label="trasa", xlabel="X", ylabel="Y"
```

```
    ))
end
```

```
Rozwiązuję TSP (n=20, seed=42)...
  Znaleziono subtour długości 3: [1, 14, 19]
  Znaleziono subtour długości 4: [1, 14, 19, 18]
  Znaleziono subtour długości 11: [1, 3, 7, 13, 10, 20, 17, 12, 18, 19, 14]
  Znaleziono subtour długości 17: [1, 9, 4, 5, 2, 15, 8, 13, 10, 7, 3, 20,
   ↪  17, 12, 18, 19, 14]
  Znaleziono subtour długości 15: [1, 9, 4, 5, 2, 15, 8, 11, 6, 16, 17, 12,
   ↪  18, 19, 14]
  Znaleziono subtour długości 14: [1, 3, 7, 20, 10, 13, 11, 6, 16, 17, 12,
   ↪  18, 19, 14]
  Znaleziono subtour długości 11: [1, 3, 7, 13, 10, 20, 17, 12, 18, 19, 14]
  Znaleziono subtour długości 11: [1, 3, 7, 13, 10, 20, 17, 12, 18, 19, 14]
Status: OPTIMAL, czas: 0.22 s
 Koszt: 3.5412
  Trasa: [1, 9, 4, 5, 2, 15, 8, 11, 6, 16, 17, 12, 20, 10, 13, 7, 3, 18, 19,
   ↪  14]
```



```julia
function find_subtour_tpp(x_vals, v_vals, n)
    visited = falses(n)

    for start in 1:n
        visited[start] && continue
        v_vals[start] < 0.5 && continue
        any(x_vals[start, j] > 0.5 for j in 1:n) || continue

        tour = Int[]
        current = start

        for _ in 1:n
            visited[current] && break
            push!(tour, current)
            visited[current] = true

            moved = false
            for j in 1:n
                if x_vals[current, j] > 0.5 && !visited[j] && v_vals[j] > 0.5
                    current = j
                    moved = true
                    break
                end
            end
            moved || break
        end

        # zwróć pierwszy subtour krótszy niż pełna trasa
        n_visited = count(v_vals .> 0.5)
        if 1 < length(tour) < n_visited
            return tour
        end
    end

    return Int[]
end
```

Stworzony problem dla pomijania miast

Constrainty: - x symetryczne - każdy przedmiot kupiony w jednym mieście - zakupy tylko w miastach odwiedzonych - jeśli miasto jest odwiedzone to ma wejście i wyjście - nie ma pętli

jedno punktowych

```
function build_tpp_lazy_model(d, cost_item_city)
    n = size(d, 1)
    k = size(cost_item_city, 1)

    model = Model(GLPK.Optimizer)
    set_attribute(model, "msg_lev", GLPK.GLP_MSG_OFF)

    @variable(model, x[i=1:n, j=1:n], Bin, Symmetric)
    @variable(model, y[item=1:k, city=1:n], Bin)
    @variable(model, v[city=1:n], Bin)

    @objective(model, Min,
        sum(d[i, j] * x[i, j] for i in 1:n, j in 1:n if i < j) +
        sum(cost_item_city[item, city] * y[item, city] for item in 1:k, city
         ↪  in 1:n)
    )

    for item in 1:k
        @constraint(model, sum(y[item, :]) == 1)
    end

    for city in 1:n, item in 1:k
        @constraint(model, y[item, city] <= v[city])
    end

    for city in 1:n
        @constraint(model, sum(x[city, j] for j in 1:n) == 2 * v[city])
        @constraint(model, x[city, city] == 0)
    end

    function subtour_elimination_callback(cb_data)
        status = callback_node_status(cb_data, model)
        status != MOI.CALLBACK_NODE_STATUS_INTEGER && return

        x_vals = callback_value.(cb_data, x)
        v_vals = callback_value.(cb_data, v)
        n_visited = count(v_vals .> 0.5)
        n_visited < 2 && return

        cycle = find_subtour_tpp(x_vals, v_vals, n)
```

```
        isempty(cycle) && return

        println("  Subtour długości $(length(cycle)) z $n_visited
         ↪  odwiedzonych: $cycle")

        S = [(i, j) for i in cycle, j in cycle if i < j]
        con = @build_constraint(
            sum(x[i, j] for (i, j) in S) <= length(cycle) - 1
        )
        MOI.submit(model, MOI.LazyConstraint(cb_data), con)
    end

    set_attribute(model, MOI.LazyConstraintCallback(),
     ↪  subtour_elimination_callback)

    return model
end
```

```
function solve_tpp_lazy(n, k; random_seed=1, min_cities=3, time_limit=120.0)
    X, Y, d, cost_item_city = generate_random_tpp_selective(n, k;
        random_seed=random_seed, min_cities=min_cities, max_cities=n-2)

    model = build_tpp_lazy_model(d, cost_item_city)
    set_time_limit_sec(model, time_limit)

    println("Rozwiązuję TPP (n=$n, k=$k, seed=$random_seed)...")
    t_start = time()
    optimize!(model)
    t_elapsed = time() - t_start

    status = termination_status(model)
    println("Status: $status, czas: $(round(t_elapsed, digits=2)) s")

    if !is_solved_and_feasible(model)
        return (
            status=status, cost=Inf, time=t_elapsed,
            X=X, Y=Y, d=d, cost_item_city=cost_item_city,
            tour=Int[], visited_cities=Int[],
            purchases=Dict{Int,Int}(),
            num_cities_visited=0, num_cities_skipped=n
        )
```

```
    end

    x_sol = JuMP.value.(model[:x])
    y_sol = JuMP.value.(model[:y])
    v_sol = JuMP.value.(model[:v])

    visited = [i for i in 1:n if v_sol[i] > 0.5]

    tour    = Int[]
    in_tour = Set{Int}()
    current = visited[1]
    push!(tour, current); push!(in_tour, current)
    while true
        nxt = nothing
        for j in 1:n
            if j   in_tour && x_sol[current, j] > 0.5
                nxt = j; break
            end
        end
        nxt === nothing && break
        push!(tour, nxt); push!(in_tour, nxt)
        current = nxt
        length(tour) >= n && break
    end

    purchases = Dict{Int,Int}()
    for item in 1:k
        for city in 1:n
            if y_sol[item, city] > 0.5
                purchases[item] = city
                break
            end
        end
    end

    return (
        status=status,
        cost=objective_value(model),
        time=t_elapsed,
        X=X, Y=Y, d=d, cost_item_city=cost_item_city,
        tour=tour,
        visited_cities=visited,
```

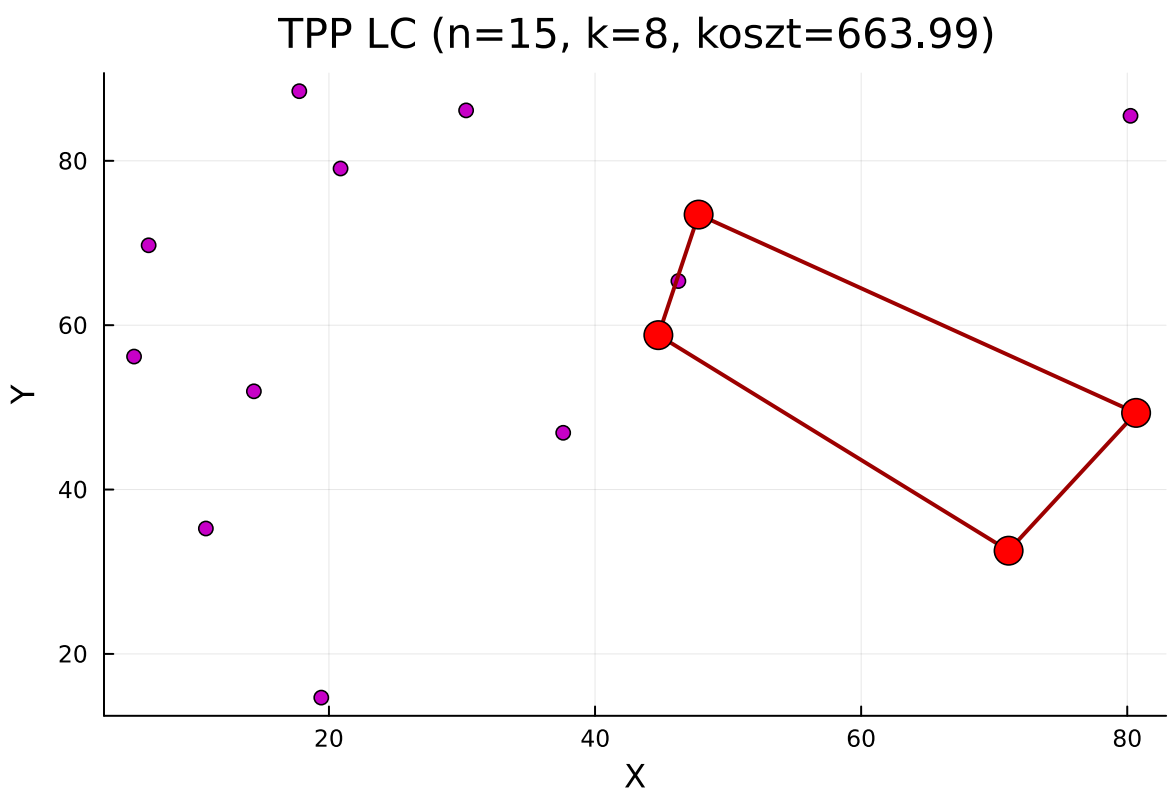
```
        purchases=purchases,
        num_cities_visited=length(visited),
        num_cities_skipped=n - length(visited)
    )
end

function print_tpp_solution(res, n, k)
    res.cost == Inf && return

    println("\n  Koszt całkowity:      $(round(res.cost, digits=2))")
    println("  Miast odwiedzonych:   $(res.num_cities_visited) / $n")
    println("  Miast pominiętych:    $(res.num_cities_skipped)")
    println("  Czas:                 $(round(res.time, digits=2)) s")
    println("  Trasa: $(res.tour)")

    println("\n  Zakupy (przedmiot → miasto → koszt):")
    for item in 1:k
        city = get(res.purchases, item, nothing)
        if city !== nothing
            cost = res.cost_item_city[item, city]
            @printf "    Przedmiot %3d → miasto %3d  (koszt: %.2f)\n" item
             ↪  city cost
        end
    end

    println("\n  Zakupy per miasto:")
    city_items = Dict{Int, Vector{Int}}()
    for (item, city) in res.purchases
        push!(get!(city_items, city, Int[]), item)
    end
    for city in sort(collect(keys(city_items)))
        items = city_items[city]
        costs = [res.cost_item_city[i, city] for i in items]
        @printf "    Miasto %3d: przedmioty %s  (suma: %.2f)\n" city items
         ↪  sum(costs)
    end
end
```

```
function plot_tpp_tour(res; title="TPP Lazy Constraints")
    p = Plots.plot(title=title, xlabel="X", ylabel="Y", legend=false)
```

```julia
    Plots.scatter!(p, res.X, res.Y, color=:gray, markersize=4)

    if !isempty(res.tour)
        tour_closed = [res.tour; res.tour[1]]  # zamknij pętlę
        Plots.plot!(p,
            [res.X[c] for c in tour_closed],
            [res.Y[c] for c in tour_closed],
            color=:blue, linewidth=2
        )
    end

    city_items = Dict{Int, Vector{Int}}()
    for (item, city) in res.purchases
        push!(get!(city_items, city, Int[]), item)
    end
    for (city, items) in city_items
        Plots.scatter!(p, [res.X[city]], [res.Y[city]], color=:red,
↪   markersize=8)
    end

    return p
end

res = solve_tpp_lazy(15, 8; random_seed=42, time_limit=60.0)
print_tpp_solution(res, 15, 8)
display(plot_tpp_tour(res; title="TPP LC (n=15, k=8, koszt=$(round(res.cost,
↪   digits=2)))"))
```

```
Rozwiązuję TPP (n=15, k=8, seed=42)...
Status: OPTIMAL, czas: 0.0 s

  Koszt całkowity:      663.99
  Miast odwiedzonych:   4 / 15
  Miast pominiętych:    11
  Czas:                 0.0 s
  Trasa: [1, 7, 3, 12]

  Zakupy (przedmiot → miasto → koszt):
    Przedmiot   1 → miasto   1  (koszt: 66.29)
    Przedmiot   2 → miasto   1  (koszt: 59.30)
    Przedmiot   3 → miasto   1  (koszt: 85.86)
    Przedmiot   4 → miasto   3  (koszt: 63.56)
```

```
    Przedmiot   5 → miasto   7  (koszt: 83.52)
    Przedmiot   6 → miasto   1  (koszt: 67.40)
    Przedmiot   7 → miasto   1  (koszt: 51.71)
    Przedmiot   8 → miasto  12  (koszt: 74.12)

  Zakupy per miasto:
    Miasto   1: przedmioty [6, 7, 2, 3, 1]  (suma: 330.56)
    Miasto   3: przedmioty [4]  (suma: 63.56)
    Miasto   7: przedmioty [5]  (suma: 83.52)
    Miasto  12: przedmioty [8]  (suma: 74.12)
```

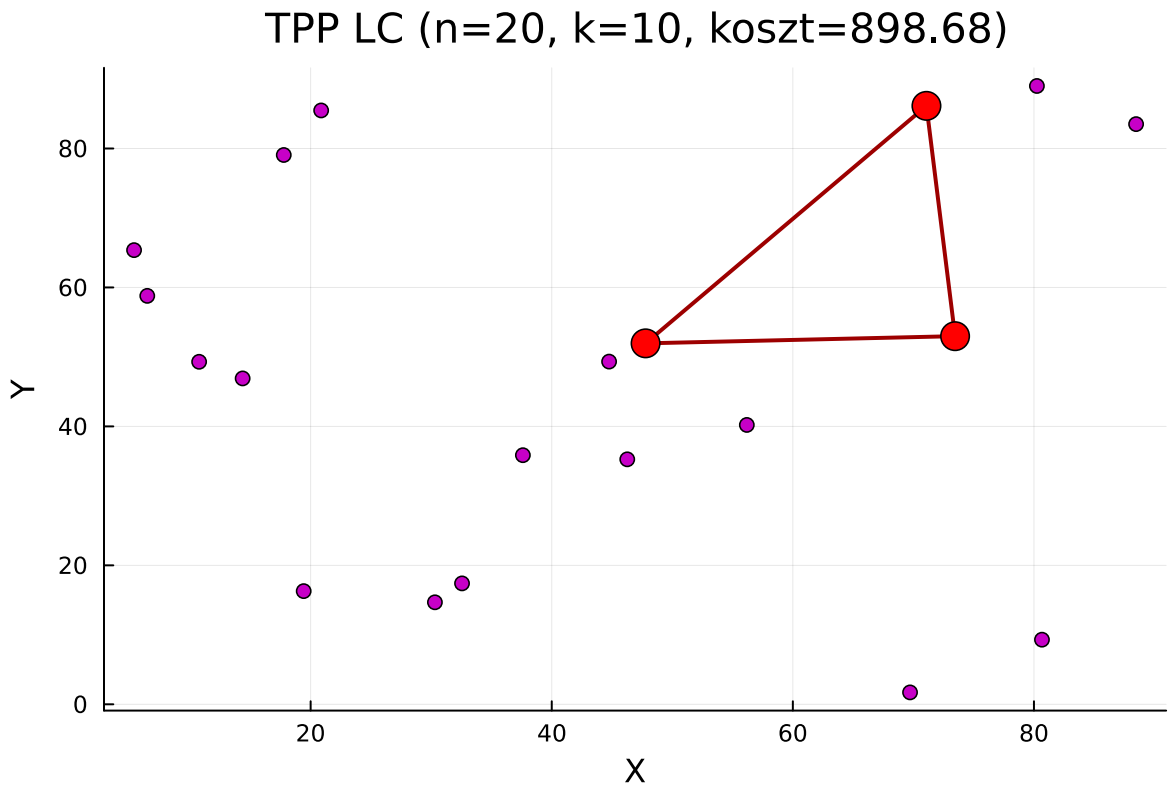



```julia
res = solve_tpp_lazy(20, 10; random_seed=42, time_limit=60.0)
print_tpp_solution(res, 20, 10)
display(plot_tpp_tour(res; title="TPP LC (n=20, k=10, koszt=$(round(res.cost,
 ↪  digits=2)))"))
```

```
Rozwiązuję TPP (n=20, k=10, seed=42)...
  Subtour długości 3 z 6 odwiedzonych: [1, 3, 18]
  Subtour długości 4 z 7 odwiedzonych: [1, 3, 20, 18]
```

```
  Subtour długości 4 z 7 odwiedzonych: [1, 3, 10, 18]
  Subtour długości 4 z 7 odwiedzonych: [1, 3, 13, 18]
  Subtour długości 3 z 6 odwiedzonych: [1, 3, 18]
  Subtour długości 4 z 7 odwiedzonych: [1, 3, 20, 18]
  Subtour długości 4 z 7 odwiedzonych: [1, 3, 10, 18]
  Subtour długości 4 z 7 odwiedzonych: [1, 3, 13, 18]
  Subtour długości 3 z 6 odwiedzonych: [1, 3, 7]
  Subtour długości 4 z 7 odwiedzonych: [1, 3, 7, 18]
  Subtour długości 5 z 8 odwiedzonych: [1, 3, 7, 20, 18]
  Subtour długości 5 z 8 odwiedzonych: [1, 3, 7, 13, 18]
  Subtour długości 6 z 9 odwiedzonych: [1, 3, 7, 13, 20, 18]
  Subtour długości 4 z 7 odwiedzonych: [1, 3, 18, 14]
  Subtour długości 3 z 9 odwiedzonych: [1, 14, 19]
  Subtour długości 3 z 9 odwiedzonych: [1, 14, 19]
  Subtour długości 3 z 9 odwiedzonych: [1, 14, 18]
  Subtour długości 3 z 9 odwiedzonych: [1, 14, 19]
  Subtour długości 3 z 9 odwiedzonych: [1, 14, 18]
  Subtour długości 3 z 9 odwiedzonych: [1, 14, 19]
  Subtour długości 3 z 9 odwiedzonych: [1, 14, 19]
  Subtour długości 3 z 9 odwiedzonych: [1, 14, 19]
  Subtour długości 3 z 9 odwiedzonych: [1, 14, 18]
  Subtour długości 3 z 9 odwiedzonych: [1, 14, 19]
  Subtour długości 3 z 9 odwiedzonych: [1, 14, 19]
  Subtour długości 3 z 9 odwiedzonych: [1, 14, 19]
  Subtour długości 3 z 9 odwiedzonych: [1, 14, 19]
  Subtour długości 3 z 6 odwiedzonych: [1, 14, 19]
  Subtour długości 4 z 7 odwiedzonych: [1, 3, 7, 18]
  Subtour długości 5 z 8 odwiedzonych: [1, 3, 7, 20, 18]
  Subtour długości 3 z 9 odwiedzonych: [1, 14, 19]
  Subtour długości 4 z 7 odwiedzonych: [1, 3, 20, 18]
  Subtour długości 3 z 6 odwiedzonych: [1, 14, 19]
  Subtour długości 3 z 9 odwiedzonych: [1, 14, 19]
  Subtour długości 4 z 7 odwiedzonych: [1, 3, 7, 18]
  Subtour długości 4 z 7 odwiedzonych: [1, 3, 20, 18]
  Subtour długości 3 z 9 odwiedzonych: [1, 18, 19]
Status: OPTIMAL, czas: 0.02 s

 Koszt całkowity:      898.68
  Miast odwiedzonych:   3 / 20
  Miast pominiętych:    17
  Czas:                 0.02 s
  Trasa: [1, 3, 18]
```

```
Zakupy (przedmiot → miasto → koszt):
  Przedmiot   1 → miasto   3  (koszt: 88.50)
  Przedmiot   2 → miasto   3  (koszt: 60.94)
  Przedmiot   3 → miasto   1  (koszt: 125.20)
  Przedmiot   4 → miasto   1  (koszt: 94.67)
  Przedmiot   5 → miasto   1  (koszt: 53.35)
  Przedmiot   6 → miasto  18  (koszt: 73.11)
  Przedmiot   7 → miasto   3  (koszt: 80.01)
  Przedmiot   8 → miasto   1  (koszt: 83.75)
  Przedmiot   9 → miasto   3  (koszt: 75.30)
  Przedmiot  10 → miasto   1  (koszt: 63.56)

Zakupy per miasto:
  Miasto   1: przedmioty [5, 4, 10, 8, 3]  (suma: 420.53)
  Miasto   3: przedmioty [7, 2, 9, 1]  (suma: 304.75)
  Miasto  18: przedmioty [6]  (suma: 73.11)
```

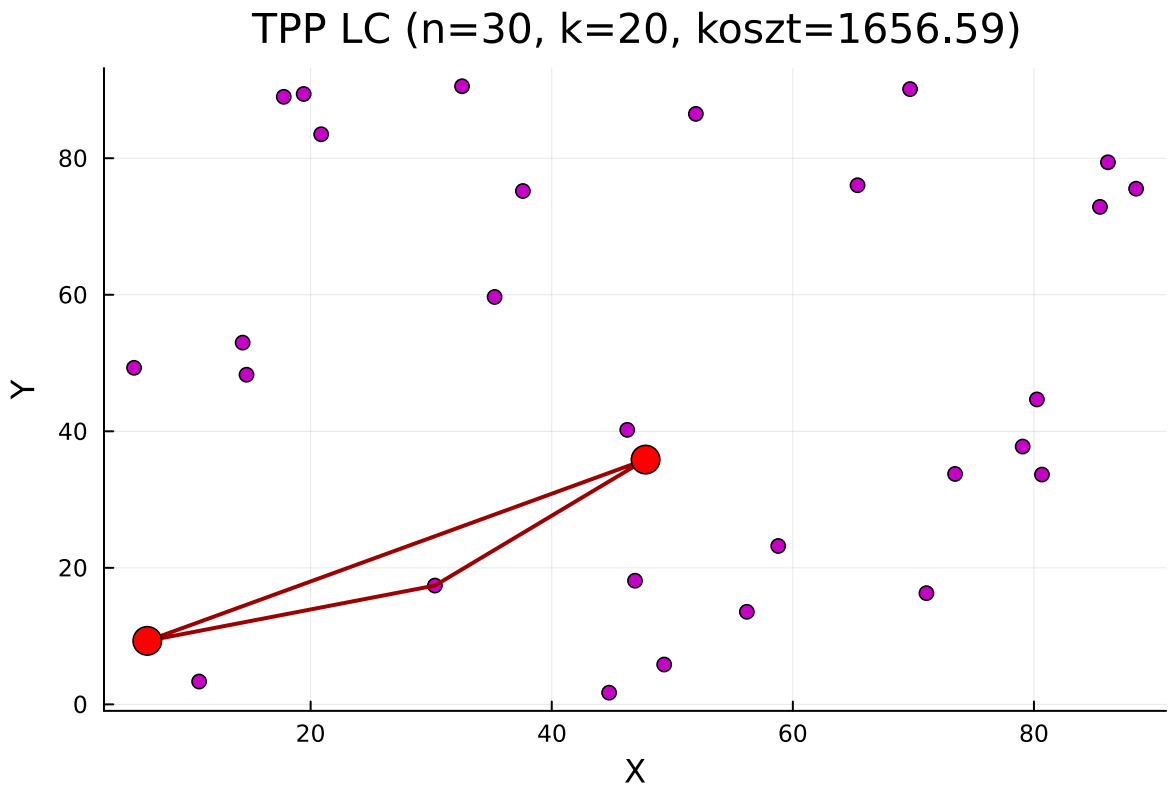



```
res = solve_tpp_lazy(30, 20; random_seed=42, time_limit=60.0)
print_tpp_solution(res, 30, 20)
```

```julia
display(plot_tpp_tour(res; title="TPP LC (n=30, k=20, koszt=$(round(res.cost,
↪  digits=2)))"))
```

```
Rozwiązuję TPP (n=30, k=20, seed=42)...
  Subtour długości 3 z 6 odwiedzonych: [1, 3, 22]
  Subtour długości 3 z 6 odwiedzonych: [1, 3, 28]
  Subtour długości 4 z 7 odwiedzonych: [1, 3, 28, 20]
  Subtour długości 4 z 7 odwiedzonych: [1, 18, 3, 28]
  Subtour długości 4 z 7 odwiedzonych: [1, 20, 3, 22]
  Subtour długości 5 z 8 odwiedzonych: [1, 20, 28, 3, 22]
  Subtour długości 5 z 8 odwiedzonych: [1, 18, 3, 22, 20]
  Subtour długości 3 z 6 odwiedzonych: [2, 6, 15]
  Subtour długości 4 z 7 odwiedzonych: [1, 22, 3, 28]
  Subtour długości 5 z 8 odwiedzonych: [1, 22, 3, 28, 27]
  Subtour długości 3 z 6 odwiedzonych: [1, 3, 20]
  Subtour długości 3 z 6 odwiedzonych: [1, 3, 18]
  Subtour długości 3 z 6 odwiedzonych: [1, 3, 12]
  Subtour długości 4 z 7 odwiedzonych: [1, 3, 24, 12]
  Subtour długości 3 z 6 odwiedzonych: [1, 20, 22]
  Subtour długości 3 z 6 odwiedzonych: [1, 3, 10]
  Subtour długości 4 z 7 odwiedzonych: [1, 10, 3, 22]
  Subtour długości 4 z 7 odwiedzonych: [1, 10, 3, 28]
  Subtour długości 4 z 7 odwiedzonych: [1, 18, 3, 22]
  Subtour długości 4 z 7 odwiedzonych: [1, 22, 3, 28]
  Subtour długości 3 z 6 odwiedzonych: [2, 6, 15]
  Subtour długości 4 z 7 odwiedzonych: [1, 22, 3, 27]
  Subtour długości 3 z 6 odwiedzonych: [1, 20, 22]
  Subtour długości 5 z 8 odwiedzonych: [1, 20, 3, 10, 22]
  Subtour długości 6 z 9 odwiedzonych: [1, 20, 28, 10, 3, 22]
  Subtour długości 4 z 7 odwiedzonych: [1, 10, 3, 20]
  Subtour długości 3 z 6 odwiedzonych: [2, 6, 15]
Status: OPTIMAL, czas: 0.04 s

 Koszt całkowity:      1656.59
  Miast odwiedzonych:   3 / 30
  Miast pominiętych:    27
  Czas:                 0.04 s
  Trasa: [2, 3, 6]

  Zakupy (przedmiot → miasto → koszt):
    Przedmiot   1 → miasto   2  (koszt: 71.88)
    Przedmiot   2 → miasto   2  (koszt: 59.54)
```

```
  Przedmiot   3 → miasto   2  (koszt: 63.11)
  Przedmiot   4 → miasto   3  (koszt: 70.85)
  Przedmiot   5 → miasto   3  (koszt: 99.26)
  Przedmiot   6 → miasto   2  (koszt: 51.00)
  Przedmiot   7 → miasto   2  (koszt: 78.90)
  Przedmiot   8 → miasto   2  (koszt: 61.37)
  Przedmiot   9 → miasto   2  (koszt: 83.77)
  Przedmiot  10 → miasto   3  (koszt: 64.57)
  Przedmiot  11 → miasto   3  (koszt: 69.08)
  Przedmiot  12 → miasto   3  (koszt: 73.01)
  Przedmiot  13 → miasto   3  (koszt: 132.04)
  Przedmiot  14 → miasto   3  (koszt: 88.91)
  Przedmiot  15 → miasto   3  (koszt: 83.04)
  Przedmiot  16 → miasto   2  (koszt: 65.64)
  Przedmiot  17 → miasto   3  (koszt: 102.99)
  Przedmiot  18 → miasto   2  (koszt: 68.05)
  Przedmiot  19 → miasto   2  (koszt: 102.73)
  Przedmiot  20 → miasto   2  (koszt: 67.11)

Zakupy per miasto:
  Miasto   2: przedmioty [16, 20, 8, 1, 19, 6, 9, 3, 7, 2, 18]  (suma:
   ↪  773.11)
  Miasto   3: przedmioty [5, 12, 17, 11, 14, 4, 13, 15, 10]  (suma: 783.74)
```




```
res = solve_tpp_lazy(50, 20; random_seed=42, time_limit=2000.0)
print_tpp_solution(res, 50, 20)
display(plot_tpp_tour(res; title="TPP LC (n=50, k=20, koszt=$(round(res.cost,
↪  digits=2)))"))
```